AOSP 进阶丛书 · 第六册 · ANDROID 13 (API 33)

PMS 与包管理

开机扫描建立包宇宙、Computer 快照读写分离、会话化原子安装五步管线、权限三层闸门与 ROM 预装正确姿势——一切组件的出生证明办公室。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：一切组件的"出生证明办公室" 2

第一章 开机扫描：包宇宙的建立 2

第二章 Computer 快照：读写分离的包宇宙 6

第三章 一次安装：会话化流水线 7

第四章 权限：三层闸门 11

第五章 卸载、多用户与查询 13

第六章 实战：ROM 预装应用的正确姿势（BSP 视角） 14

第七章 设计决策（十一问） 15

第八章 深水区：存储配额、多 APK 与备份恢复 16

第九章 补全：APK 签名体系、Intent 匹配与安装后布局 17

附录 A 观测实操 18

附录 B 快问快答（16 条） 19

附录 C 自检题 19

附录 D 动手实验室（半天量） 19

第十章 构造器与 systemReady：包宇宙的开机两幕 20

第十一章 会话三阶段全链：commit、verify 与 install 24

第十二章 权限账本：持久化、恢复与升级重算 30

第十三章 APEX：apexd、ApexManager 与包宇宙的双轨扫描 36

教科书习题集（分级） 42

术语表（按拼音排序） 46

排障手册：PMS 与包管理问题定位 48

附录 E：PMS 核心机制深度速查表 50

本册速览

开机扫描建包宇宙（两本账：内存表+packages.xml）
Computer 快照读写分离 · 会话化原子安装五步 · 失败码词典
权限三层闸门（声明/授予/AppOps） · ROM 预装四件套
深水区：installd 配额/split/备份 · 补全：签名 v1-v4/Intent 匹配
存储与加密衔接：vold/FBE/keystore2（书11）

前言：一切组件的"出生证明办公室"

前四册里每个组件的启动都要"问 PMS 要地址"（queryIntentActivities、resolveActivity…）。本册终于解剖这座图书馆本身：**开机时它如何把全机 APK 读成一张包宇宙大表；一次安装如何原子地改写这个世界；权限这座三层闸门如何放行或拦截；dex 优化为何在 Android 13 交给 ART 模块。**

体例不变：源码逐段注释 + 逐段精读。本树（Android 13）PMS 有两处与旧教材显著不同的架构演进——**Computer 快照（读写分离）**与**安装/dexopt 职责外迁（InstallPackageHelper/ART 模块）**，本册会反复对照"旧世界"以防读者拿着旧地图找新路。

0.1 新旧世界对照表（读旧教材/旧博客前必看）

概念	旧世界（≤11）	本树（13）
安装执行	VerificationParams/ASEC	InstallPackageHelper（聚合器）+ InstallParams.installStage
查询	PMS 大锁内直接读	Computer 不可变快照（读无锁）
dexopt	installd 直接跑 dex2oat	ART 模块 artd 执行，PMS 只管策略
ActivityStackSupervisor	——	ActivityTaskSupervisor（书4 提及的更名）
解析	PackageParser	ParsingPackageUtils + AndroidPackage（pkg.parsing/）
root	隐式 root 用户	显式（多用户语义收紧）

逐段精读：读任何 2019-2022 年的 PMS 文章，先拿这张表换算名词，再看本树源码——**名词对不上的地方不是你错了，是架构走了。**

第一章 开机扫描：包宇宙的建立

> **设计洞察 · 学习目标**
> 1. 复述 PMS.main 到 writeLPw 的开机扫描全序与分区信任序；2. 说出"两本账"（内存 mPackages/磁盘 packages.xml）的对账规则与自愈路径；3. 走读 scanSystemDirs 的五次 scanDirTracedLI 与各自的信任语义；4. 用 boot_progress 事件与 logcat 量化一次真实开机扫描



图 1 PMS 启动扫描：入口、分区扫描顺序、两本账（内存表+packages.xml）

1.1 入口与两本账（逐段注释）

java frameworks/.../pm/PackageManagerService.java:1411 起（骨架+讲解注释）

```
public static Pair<PackageManagerService, IPackageManager> main(Context context, ...) {
// 讲解注释①：PackageManagerService 构造器是"重活总汇"：
// Settings.readLPw（Settings.java:2938）读 packages.xml——
// 上次关机时的包注册表（版本/权限/用户态/签名指纹）。
// 讲解注释②：Computer 快照起步——13 的核心改造：
// 所有查询走不可变快照（computer(...)），写操作持锁后重建快照。
// 读多写少的包宇宙从此"读无锁"。
// 讲解注释③：InitAppsHelper 扫描全部分区（下节）；
// 扫描结果与 Settings 对账：删掉"盘上没了"的，标灰"升级未完成"的。
// 讲解注释④：writeLPw 落盘 + 对外发布（IPackageManager binder 就绪，
// SystemServer 的 Bootstrap 批才放行——书2 service-deps 图的 PMS 节点）。
}
```

逐段精读：①的"对账"解释了拔卡/损坏 APK 的自愈：盘上消失 → PackageSetting 删除；hash 不符 → 重装标记。**注册表（账）与文件（物）双轨，开机必对账**——数据库思维的系统级应用。

1.2 分区扫描：顺序即权限（InitAppsHelper 逐段注释）

java pm/InitAppsHelper.java:308-342 scanSystemDirs（骨架+讲解注释）

```
// 讲解注释①：scanSystemDirs(:308) 统管只读分区扫描，顺序即信任序：
// scanDirTracedLI(:325) framework 目录（framework-res 等）最先入库——
// 后续一切包依赖它的资源与共享库。
// 讲解注释②：循环各分区（system/vendor/product/system_ext）：
// :337 partition.getPrivAppFolder()（priv-app，特权由 privapp-permissions
```

```
// 白名单二次裁定）→ :342 partition.getAppFolder()（普通 app）。
// 讲解注释③：:320 partition.getOverlayFolder()（在 app 之前先收 RRO
// 资源覆盖层，供版本匹配）。
// 讲解注释④：只读分区扫完，:248 getAppInstallDir()（/data/app）最后对账：
// 用户装的可覆盖系统同包（升级语义），与 packages.xml 增量核对。
```

扫描顺序=优先级基线：framework → system(priv→normal) → vendor → data。同包名高版本胜出（升级），签名不符则拒绝覆盖——这是"系统 App 静默升级"的机制底座，也是恶意 App 冒名系统包的封印。

> **源码锚点 · 第一章锚点**
>
> PackageManagerService.java:1411 main / Settings.java:2938 readLPw / :175 类定义
InitAppsHelper.java:248/320/325 三处 scanDirTracedLI
解析器：pm/pkg/parsing/ParsingPackageUtils（APK→ParsingPackage→AndroidPackage）

1.3 单个 APK 的入库之旅（逐段注释）

java pm/InitAppsHelper→ParsingPackageUtils（骨架+讲解注释）

```
// 讲解注释①：scanDirTracedLI 遍历目录每个 .apk：
// ParsingPackageUtils.parsePackage → ApkLite（轻解析：只要 manifest 头）
// ——先看"值不值得全量解析"（版本/签名初判），省开机时间。
// 讲解注释②：全量解析 ParsingPackage：四大组件逐个读进内存对象
//  （Activity/Service/Receiver/Provider + intent-filter 列表）；
// 权限声明（uses-permission/permission 自定义）一并入库。
// 讲解注释③：签名校验：apksigner 验签 → PackageSignatures 指纹记账
//  （升级一致性与 platform 签名特权判定的依据）。
// 讲解注释④：建索引：组件按 intent-filter 写入倒排表
//  （mActivities/mServices/mReceivers/mProviders + queryIntent 快速通道），
// 供第四册所有"问地址"消费。
// 讲解注释⑤：设置默认 user 状态（enabled），priv-app 特权白名单比对。
```

逐段精读：①的"轻解析先行"是开机性能的关键取舍——全量解析（资源表/代码段偏移）推迟到真正需要时；②与④解释了 queryIntentActivities 为什么是 O(filter 命中) 而非全表扫描。**包宇宙的本质=以 manifest 为 schema 的内存数据库**，扫描就是全量导入。

1.4 packages.xml 字段速查（Settings 账本）

字段	含义	谁写
name/codePath	包名与 apk 落点（v~~随机目录）	安装五步第 2 步
versionCode/tsTime	版本与更新时间	安装/升级
signatures/cert	签名指纹	扫描③
perms（granted/denied）	权限三态（per-user）	授予/撤销路径
pkgFlags/privateFlags	SYSTEM/PRIVILEGED 标记	扫描分区归属

字段	含义	谁写
enabled/components enabled	包与组件级启停	pm enable/disable
installSource	谁装的（initiator/installer 包名）	会话提交时
firstInstallTime/lastUpdateTime	审计三兄弟	安装五步

观测：`adb shell su 0 cat /data/system/packages.xml | head -50`（userdebug）。**看一次真实文件胜过读十遍字段表。**

1.5 PMS 扫描完整源码走读（教科书级逐步分解）

第①段：从 SystemServer 调用 PMS.main

java frameworks/base/services/java/com/android/server/SystemServer.java:1233（骨架+注释）

```
// 讲解注释①：PMS.main 在 Bootstrap 批被调用——
// 书2 §七的 Bootstrap 批列表中，AMS 之后、phase 100 之后
Pair<PackageManagerService, IPackageManager> pmsPair =
PackageManagerService.main(mSystemContext, installer,
domainVerification, mFactoryTest != FactoryTest.FACTORY_TEST_OFF,
mOnlyCore); // mOnlyCore：安全模式下只扫 core 包
// 讲解注释②：mOnlyCore 在安全模式（Safe Mode）时为 true——
// 只加载核心系统包，跳过所有普通 App——"安全模式最小集"
```

第②段：PMS 构造器逐段（PackageManagerService.java）

java pm/PackageManagerService.java 构造器（骨架+讲解注释）

```
public PackageManagerService(Context context, Installer installer,
boolean factoryTest, boolean onlyCore) {
// 讲解注释③：Settings.readLPw（Settings.java:2938）
// 读 packages.xml——上次的包注册表（版本/权限/签名指纹/路径）
// 对账：删掉"盘上没了"的包、标灰"升级未完成"的包
// 讲解注释④：mSettings.mResolvedPermissions → 权限树初始化
// 讲解注释⑤：scanDirTracedLI 扫描（InitAppsHelper :248/:320/:325/:337/:342）
// 顺序：framework → priv-app → app → vendor → product → data
// 讲解注释⑥：writeLPw 写回 packages.xml（先写临时文件→原子 rename）
}
```

第③段：扫描单个 APK 的全流程

text ParsingPackageUtils.parsePackage 的五步（读码归纳）

```
步骤 1：ApkLite 轻解析（读 ZIP 中央目录，只要 manifest 头）
→ 值不值得全量解析？（版本对比/签名初判）
步骤 2：全量解析 → ParsingPackage 对象
→ 读 AndroidManifest.xml 全部标签
→ 四大组件 + intent-filter + 权限声明 + 共享库
步骤 3：签名校验 → ApkSignatureVerifier（core/java/android/util/apk/）
→ v2/v3（SigningBlock）→ v1（META-INF 兜底）
步骤 4：建索引 → intent-filter 写入倒排表
→ mActivities/mServices/mReceivers/mProviders
→ queryIntent 的 O(filter命中) 查询基础
```

```
步骤 5：PackageSetting 创建 → Settings 立项
→ priv-app 特权白名单比对（privapp-permissions-*.xml）
```

第④段：scanDirTracedLI 逐行注释

java pm/InitAppsHelper.java:308-342（骨架+讲解注释）

```
private void scanSystemDirs(PackageParser2 packageParser, ...) {
// 讲解注释①：:325 framework 目录最先扫——
// framework-res.apk 等最先入库（后续包依赖它的资源/共享库）
scanDirTracedLI(frameworkDir, null, ..., packageParser, executorService);
// 讲解注释②：:320 overlay 目录在 app 之前——
// RRO 资源覆盖层先收（版本匹配优先级高于 app）
scanDirTracedLI(partition.getOverlayFolder(), ..., packageParser, ...);
// 讲解注释③：:337 各分区 priv-app → 特权包（白名单二次裁定）
scanDirTracedLI(partition.getPrivAppFolder(), ..., packageParser, ...);
// 讲解注释④：:342 各分区普通 app → 系统应用
scanDirTracedLI(partition.getAppFolder(), ..., packageParser, ...);
// 讲解注释⑤：:248 最后扫 /data/app —— 可覆盖系统同包
scanDirTracedLI(mPm.getAppInstallDir(), ..., packageParser, ...);
}
```

设计洞察 · 逐段精读：扫描顺序为什么是"先系统后数据"

优先级=信任级。system 分区的包在 PMS 眼里"更可信"——data 分区的同包名高版本 APK 可以覆盖 system 版本（升级语义），但签名必须一致（信任链不中断）。**信任边界=分区边界**，这是 PMS 扫描的核心设计原则。

第二章 Computer 快照：读写分离的包宇宙

设计洞察 · 学习目标

1. 说出 Computer 快照"改-建-换"三步与 volatile 引用交换的可见性语义；2. 解释为什么读路径可以完全无锁而写路径必须持 mLock；3. 对比快照读与旧世界大锁读的并发差异；4. 用"安装中 dumpsys 读到旧版本"的实验验证快照边界（附录 D 实验三）

Android 13 前的 PMS 是"一把大锁管所有查询"；13 起 `Computer`（`pm/Computer.java`）成为查询世界：每个写操作（安装/卸载/权限/禁用）在锁内完成**变更+重建快照**，读者永远拿一致的旧快照，写完原子换引用。收益：并发查询不再互相排队（AMS/WMS 每秒千次 queryIntent 不再与安装互卡）；代价：写路径多一次快照构建（安装本就低频）。

2.1 Computer 逐段注释（换引用的瞬间）

java pm/PackageManagerService/Computer（骨架+讲解注释）

```
// 讲解注释①：构造：把 mSettings+mPackages 等视图打包成不可变对象；
// mComputer 字段 volatile——读者永远看到"旧快照"或"新快照"之一，
// 没有"装了一半的包宇宙"（可见性与原子性的教科书应用）。
// 讲解注释②：查询入口族：queryIntentActivitiesInternal/
// checkUidPermission/getApplicationInfo... 全部只读快照，
```

```
// 不再 synchronized(PMS.this)——AMS 的千级 QPS 从此不与安装互锁。
// 讲解注释③：写路径：PMS 写方法在 mWriteLock 内改 mSettings/mPackages
// → new Computer(...) 重建 → 赋回 mComputer（原子换引用）→ 放锁。
// 安装低频，重建快照的毫秒级成本被写频率摊薄。
```

逐段精读：③的"改-建-换"三步让读写并发度拉满，代价是**同一次安装内的多次读可能看到旧快照**——但安装的可见性本来就该以"广播+完成回调"为界，这正是语义与实现的精确对齐。

先修要求与学习路线

先修：已读前三册（架构/启动/Binder）。

读完本书你将能够：

- 追踪PMS 与包管理的核心机制的完整调用链
- 解释关键设计决策的"为什么"
- 使用 dumpsys 定位本领域问题
- 回答相关面试题（书7 对应章节）

设计洞察 · 逐段精读：为什么 PMS 最需要这个改造

PMS 是全机读频率最高的服务（每次 startActivity/Service/bind/sendBroadcast 都要 queryIntent），而写极稀疏（装/卸）。读写比万比一的场景，读写分离是教科书式解法——与 Binder 的"注册表查询缓存"（书3 §7.1）、system_server 的 LocalServices 同属一族：**把"热读冷写"的服务从锁里解放出来**。

第三章 一次安装：会话化流水线

设计洞察 · 学习目标

1. 默画会话化安装五步（staging→校验→落盘→dexopt→记账广播）与每步的原子性保障点；2. 解释 v~~随机目录防的是哪类攻击；3. 背出 INSTALL_FAILED_* 失败码到管线步骤的映射（§3.5）；4. 说明 dexopt 职责外迁 ART 模块的动机与 installd 的退守

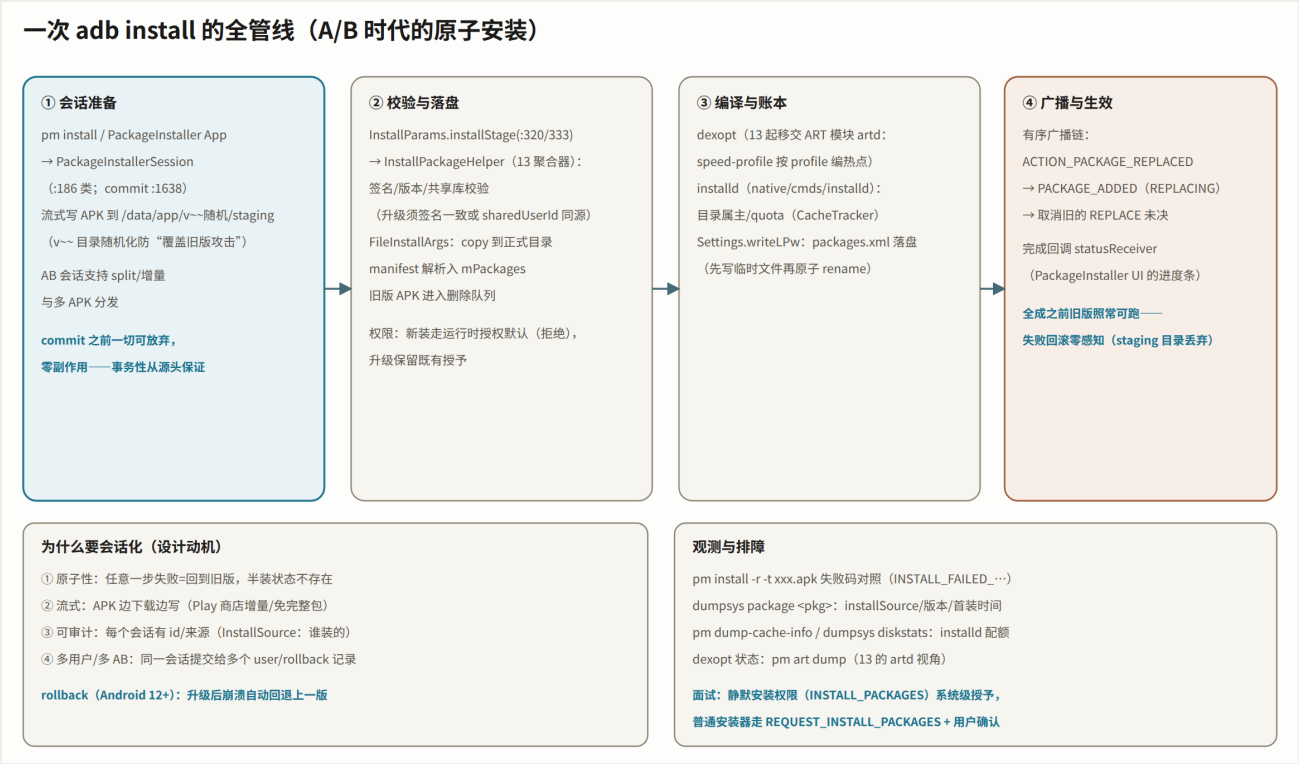


图 2 adb install 全管线：会话准备→校验落盘→编译记账→广播生效，含回滚与观测

3.1 PackageInstallerSession 逐段注释

java pm/PackageInstallerSession.java:186/1638（骨架+讲解注释）

```
public class PackageInstallerSession extends IPackageInstallerSession.Stub {
// 讲解注释①：会话三态：数据写入（staging 目录 /data/app/v~~随机/）
// → commit(:1638) 才进入管线；此前随时可放弃，零副作用。
// 讲解注释②：v~~ 目录随机化：防"预放旧版再升级"的路径猜测攻击。
// 讲解注释③：流式写入：Play 增量下载/ADB stream 都写同一会话，
// 支持多 APK（base+split）一次提交。
// 讲解注释④：commit → InstallParams.installStage(:320/333)
// → InstallPackageHelper（13 的安装聚合器，旧教材的 VerificationParams/
// ASECInstallArgs 等已在此收编）。
}
```

3.2 落盘五步与广播

校验（签名/版本/sharedUserId 同源）→ FileInstallArgs 拷贝到正式目录 → manifest 解析入 mPackages（Computer 换新快照）→ 旧版 APK 入删除队列 → Settings.writeLPw（**临时文件+原子 rename**，掉电不烂账）。生效信号是**有序广播链**：PACKAGE_REPLACED → PACKAGE_ADDED(REPLACING) → ...，MA（Managed Profile）与备份服务等按序感知。全链任何一步失败：staging 丢弃，旧版全程在线——**用户永远感知不到"装了一半"**。

3.3 dexopt：为什么搬去 ART 模块

13 起 dex 编译的执行者从 installd 移交 **artd**（com.android.runtime APEX 内，Mainline 可升级）：PMS/ArtManagerService 只做策略（何时/何种 profile），ardt 负责执行 dex2oat（书1 §8.1 三级执行的落点）。动机：**编译器与运行时必须同版本进化**（profile 格式/ABI 变更），分家即版本漂移。installd 退守存储管家：目录属主、配额（CacheTracker）、crate 管理。

3.4 广播链与回滚（12+）

安装收尾的有序广播不是"通知一下"那么简单，链条的每一环都有系统行为挂钩：

text PACKAGE_REPLACED 广播链（有序，按注册序）

```
1 PMS 自己的接收器：更新运行时权限态（升级保留授予）、备份记账
2 备份服务：新版本纳入备份集
3 Managed Profile/设备策略：工作资料内同步判定
4 备份/输入法等系统组件：默认应用重指向（若被替换的是键盘等）
5 最后才是第三方接收者（你 App 里的 PACKAGE_ADDED 监听）
```

回滚（Rollback，Android 12+）：升级时 `RollbackManager` 记录旧版 staging 与数据快照指针；新版崩溃率超阈值（Watchdog 判定）自动回退——安装管线的"后悔药"，本质是给会话多挂一个反向事务。`pm uninstall-staged/dumpsys rollback` 可观测。

3.5 安装失败码全表（排障词典）

失败码	含义	常见诱因
INSTALL_FAILED_ALREADY_EXISTS	同名已装且非覆盖	-r 缺失
INSTALL_FAILED_INVALID_APK	解析失败	损坏/manifest 非法
INSTALL_FAILED_INVALID_URI	路径不可达	URI 权限
INSTALL_FAILED_INSUFFICIENT_STORAGE	空间不足	/data 满
INSTALL_FAILED_DUPLICATE_PACKAGE	同名同版本已存在	重复提交
INSTALL_FAILED_NO_SHARED_USER	sharedUserId 协商失败	目标 uid 冲突
INSTALL_FAILED_UPDATE_INCOMPATIBLE	签名不一致	覆盖安装换了签名
INSTALL_FAILED_MISSING_SHARED_LIBRARY	依赖库缺失	uses-library 未声明
INSTALL_FAILED_CONTAINER_ERROR	存储容器错	vold/卷异常

失败码	含义	常见诱因
INSTALL_FAILED_VERIFICATION_FAILURE	校验拒绝	验证器/策略拦截

逐段精读：这些码不是异常文案，是**管线每一步的"门牌号"**——拿到码先回 §3.2 对应哪一步，再下钻那一门（解析/校验/存储/签名），排障路径立即收窄到单点。

3.6 安装管线完整源码走读（教科书级逐步分解）

第①段：PackageInstallerSession 准备

java pm/PackageInstallerSession.java:186/1638（骨架+讲解注释）

```
public class PackageInstallerSession extends IPackageInstallerSession.Stub {
// 讲解注释①：每个 install 命令对应一个 Session（有 ID）
// 会话三态：数据写入→commit→完成/失败
// 数据写入（staging 目录）：/data/app/v~~<随机32位>/staging
// 随机目录防"预放旧版再升级"的路径猜测攻击
public void commit(@NonNull IntentSender statusReceiver, boolean forTransfer) {
// :1638
// 讲解注释②：commit 触发校验→copy→dexopt→记账→广播 五步
// 此前一切可放弃，零副作用（事务性）
// 讲解注释③：InstallParams.installStage(:320)
// → InstallPackageHelper（13 的安装聚合器）
}
}
```

第②段：校验（签名/版本/共享库）

text InstallPackageHelper 的校验链（读码归纳）

```
① 签名一致：新旧版本签名必须匹配（v2/v3 指纹对比）
→ 不一致 → INSTALL_FAILED_UPDATE_INCOMPATIBLE
② 版本码：新 >= 旧（降级拒绝）
→ 覆盖安装时校验
③ 共享库：uses-library 声明的库是否可用
→ 缺失 → INSTALL_FAILED_MISSING_SHARED_LIBRARY
④ 权限冲突：sharedUserId 相同则签名必须一致
```

第③段：落盘（copy + dexopt）

text 安装五步的物理操作

```
① copy：staging → 正式目录 /data/app/v~~<hash>/c~~<hash>/
（APK 的最终位置，包含 base.apk 和 split_*.apk）
② 解压：lib/<abi>/*.so（extractNativeLibs=false 时不解压）
③ dexopt：13 起移交 ART 模块的 artd（书6 §三.3）
→ speed-profile 编译（只编热点方法）
④ 设置目录属主：chown 到目标 uid → vold 补 SELinux 标签
⑤ Settings.writeLPw：packages.xml 落盘（先写临时→rename 原子）
```

第④段：广播链

text 安装完成的有序广播链（按优先级）

```
1. PMS 自己：更新运行时权限态、备份记账
2. 备份服务：新版本纳入备份集
3. 工作资料/设备策略：同步判定
4. 系统组件：默认应用重指向（若替换的是键盘）
5. 最后才是第三方接收者（你 App 里的 PACKAGE_ADDED）
```

第⑤段：失败回滚

text 任何一步失败 → staging 目录丢弃 → 旧版全程在线

```
// 讲解注释：用户永远感知不到"装了一半"——
// 这是 A/B 安装模型的原子性保证
// 对比老世界（Android 5 前）：install 中断 → 系统坏 → 必须 re-flash
```

设计洞察 · 逐段精读：为什么要会话化

三个维度：**原子性**（staging 隔离，任意失败=回到旧版）、**流式**（APK 边下载边写，Play 增量/免完整包）、**可审计**（InstallSource 记录"谁装的"）。**会话化=把安装从"命令"变成"事务"**——这是包管理从粗放到精炼的标志。

第四章 权限：三层闸门

设计洞察 · 学习目标

1. 复述权限检查三层闸门（声明/授予/AppOps）与各自的撤销通道；2. 说清 priv-app 特权"分区+白名单"双条件与 DefaultPermissionGrantPolicy 的角色授予；3. 描述 PermissionController 与 PMS 的"界面与账本"分工；4. 用 appops/pm 命令现场验证权限三态



图 3 权限检查调用栈与运行时权限三态机、默认授予、观测

4.1 检查栈逐段注释

text 权限检查栈（对照 pms-perm 图）

```
context.checkPermission(PERM, pid, uid) ← App/服务侧入口
→ PMS(Computer).checkUidPermission ← 快照查询
→ PermissionManagerService.checkUidPermission(:226→:238)
→ 三层判定：
① 安装时声明过吗？（manifest requestedPermissions）
② 运行时授予态？（per-user 三态机）
③ AppOps 即时开关？（用户"仅这次"/撤销）
→ 特例通道：platform 签名、sharedUserId、root/system uid 白名单
```

逐段精读：③是常被忽略的一层——**"授予"不等于"每次都过"**（AppOps 可在使用时再拦），相机的"仅在使用期间允许"即三层联防。对照 SELinux（书1 第十一章）：权限管"应用语义"，SELinux 管"进程行为"，两墙独立不互代。

4.2 默认授予与特权白名单

预装应用的"出厂即有权限"由 `DefaultPermissionGrantPolicy` 按角色（拨号/短信/浏览器…）授予；priv-app 的特权权限必须同时出现在 `privapp-permissions-com.xxx.xml` 白名单——**特权=分区位置+白名单双条件**，缺一构建期即拒（`host_configverification`）。升级时既有授予态保留（`PACKAGE_REPLACED` 路径不清账）。

4.3 权限对话框的真实路径

`requestPermissions` 并不是 App 直接弹窗：`Activity.requestPermissions` → `PackageManager.requestPermissionAssets…` 最终进 **PermissionController**（独立 APK，Mainline 模块）——`grantRequest` 走回 PMS 的 `grantRuntimePermission`。**UI 与账本分离**：对话框属 Controller（可独立升级换设计），授予动作仍是 PMS 单点记账。这正是 Mainline 化的典型手法：**把"策略与界面"搬进模块，把"事实与账本"留在系统**。

特殊权限通道：`SYSTEM_ALERT_WINDOW` 走设置特批页（`ACTION_MANAGE_OVERLAY_PERMISSION`）；`ACCESS_BACKGROUND_LOCATION` 强制二段授权（先前台后后台）；`BIND_DEVICE_ADMIN` 等签名级权限连弹窗资格都没有——**对话框本身就是一层权限分级的 UI 体现**。

4.4 权限三层闸门完整代码走读（教科书级逐步分解）

第一层：安装时声明（静态检查）

xml App 的 AndroidManifest.xml

```
<uses-permission android:name="android.permission.CAMERA" />
```

java PMS 侧的处理（pm/PermissionManagerService.java 汇总）

```
// 讲解注释①：uses-permission 声明被 ParsingPackage 解析入库
// → requestedPermissions[] 数组（属于 AndroidPackage 对象）
// → 每次系统启动时重新比对：新权限默认 deny（运行时权限）
```

```
// → 新增的签名级权限需检查签名是否与声明方匹配
```

第二层：运行时授予（用户意志）

java PermissionManagerService.java:226 → :238

```
private int checkUidPermission(int uid, String permName) {
// 讲解注释②：入口方法——所有权限检查的汇聚点
// 先查权限委托（Delegate），再走默认实现
return mPermissionManagerServiceImpl.checkUidPermission(uid, permName);
}
// PermissionManagerServiceImpl.checkUidPermission 的内部逻辑：
// 讲解注释③：三层判定按序执行：
// a) 安装时声明过吗？（requestedPermissions 包含此权限？）
// b) 运行时授予态？（PackageUserState 中 granted=true？）
// c) AppOps 即时开关？（AppOpsManager.checkOp()）
// → 三层全过 → PERMISSION_GRANTED / 否则 → PERMISSION_DENIED
```

第三层：AppOps 即时控制

java AppOpsManager 的角色（讲解注释）

```
// 讲解注释④：AppOps 是"使用审计+即时开关"——
// b) 记录每次使用的次数/时长/时间戳
// c) 运行时可以动态拦截（如"仅使用期间允许"= 超时自动回 deny）
// d) 与运行时权限的关系：权限是"门钥匙"，AppOps 是"门卫日志"
```

权限三层闸门完整链

text App 调用 camera.open() 时的权限检查链

```
1. App 代码调 camera.open()
2. CameraManager → CameraService（书12 §二）
3. CameraService.enforceCallingPermission（书3 §五·身份切换）
→ Context.checkPermission(CAMERA, pid, uid)
4. → PMS.checkUidPermission（:226→:238）
→ 三层判定（声明/授予/AppOps）
5. → 通过 → CameraService 返回 CameraDevice
→ 失败 → SecurityException 或 CameraAccessException
// 讲解注释⑤：SELinux 在更底层管"进程能不能连 camera_service"
// 权限层管"App 语义上许不许可"——两道独立的墙
```

> **设计洞察 · 逐段精读：为什么权限要三层而不是一层**
>
> 变更频率不同：声明极少变（静态）、授予偶尔变（用户操作）、AppOps 高频变（使用中动态调整）。**把变更频率不同的策略分层存储**，每层可独立缓存和撤销——这是"按变更频率分治"的设计直觉。

第五章 卸载、多用户与查询

设计洞察 · 学习目标

1. 说出卸载的清账顺序与 PACKAGE_REMOVED 有序链语义；2. 解释"包本体一份、PackageUserState per-user"的多用户记账模型；3. 列出查询五入口家族与 Computer 组件倒排索引的关系；4. 用 pm install/uninstall/disable --user 验证 per-user 语义

- 卸载：`deletePackage` → FileInstallArgs 清目录 → Settings 删账 → `PACKAGE_REMOVED` 有序链（含 `EXTRA_REMOVED_FOR_ALL_USERS` 语义）。
- 多用户：包本体一份，`PackageUserState` per-user 记账（enabled/distraction/授权）——`pm install --user` 的机制底座；工作资料（Managed Profile）再叠隔离。
- 查询的五个入口家族：queryIntentActivities/Services/Receivers/ContentProviders/resolveActivity——全部走 Computer 的**组件倒排索引**（intent filter 注册表），这是第四册每次 startActivity"问地址"的索引结构。

第六章 实战：ROM 预装应用的正确姿势（BSP 视角）

设计洞察 · 学习目标

1. 按分区选择预装位置（system/priv-app/product/data）并说出代价差异；2. 写出 Android.bp 特权声明与 privapp-permissions 白名单的配合；3. 用 default-permissions/*.xml 给预装应用出厂授权；4. 用 RRO 覆盖资源而不改框架代码

系统开发常任务"把一个 App 塞进 ROM 且给对权限"，标准路径四条：

- **放哪**：/system/app（普通系统应用）、/system/priv-app（特权，配 privapp-permissions 白名单）、/product/app（产品定制，OTA 不随 system 升级）、/data/app（伪预装，可卸载）。
- **怎么声明**：`Android.bp` 里 `android_app { name:..., privileged: true, ... }`（Soong 视角，书1 §7.2）；证书选 platform（等同系统签名）或共享证书。
- **默认权限**：`DefaultPermissionGrantPolicy` 的例外名单文件 `etc/default-permissions/*.xml`。
- **资源与配置定制**：RRO overlay（`device/<oem>/<product>/rro/`）覆盖框架资源——改默认值不改框架代码的官方姿势。

设计洞察 · 逐段精读：预装三问

能不能卸？（system 不可、product 视配置、data 可）；能不能静默升级？（同签名覆盖安装即可）；能不能要任意权限？（priv 白名单过不了构建期校验——**特权是协商出来的不是声明出来的**）。这三问几乎覆盖厂商接入的全部合规审查点。

5.1 多用户与工作资料的语义表

操作	效果	命令
install --user 0	只装给主用户	pm install --user 0

操作	效果	命令
install --user 10	装给副用户（工作资料）	同上换 id
uninstall --user 10	仅该用户消失（本体保留）	pm uninstall --user 10
enable/disable --user	per-user 启停	pm disable-user --user 10
默认应用 per-user	拨号/键盘可各用户不同	RoleManager 按用户记账

工作资料（Managed Profile）= 副用户 + DevicePolicy 沙箱 + 独立锁屏挑战——它是全家桶隔离（通知/权限/VPN 全套 per-profile），不只是"另一个用户"。PMS 的 per-user 记账（PackageUserState）是这一切的地基：**包是一等公民，用户是包的属性**。

第七章 设计决策（十一问）

设计洞察 · 学习目标

1. 对十一问各自给出"一句话答案+源码证据"的双层回答；2. 把每问对应到前六章的具体机制小节并互为印证；3. 练习用"变更频率/信任边界/原子性"三个设计直觉推导同类新问题

- **为什么扫描顺序固定且 system 在前？** 升级覆盖与签名信任的方向性；framework 资源是依赖根。
- **为什么账（Settings）与物（APK 文件）分离？** 掉电/损坏可对账自愈；安装原子性由"先物后账再广播"三段保证。
- **为什么 Computer 快照？** 热读冷写服务的读写分离，安装不再卡查询。
- **为什么会话化安装？** 原子性+流式+可审计（InstallSource 记录"谁装的"）。
- **为什么 dexopt 移交 ART？** 编译器与运行时同版本进化，Mainline 可独立升级。
- **为什么权限三层闸门？** 声明（静态）+授予（用户意志）+AppOps（即时控制）对应三种变更频率，各层独立缓存与撤销。
- **为什么权限 UI 独立成 PermissionController 模块？** 界面高频迭代、账本必须稳定——Mainline 分界线画在"策略可变/事实不变"处。
- **为什么 rollback 只对 staged 会话开放？** 回滚需要原子反向事务，普通安装没有 staging 目录做物质基础。
- **为什么 enabled 状态分 per-user 与组件两级？** 用户可见语义（禁用某 App）与系统语义（禁用某接收器省电）是两种生命周期，合并记账会互相误伤。
- **为什么静默安装是系统级权限？** 安装=写入代码执行权，等同系统信任边界外扩——必须有签名级把关。

第八章 深水区：存储配额、多 APK 与备份恢复

设计洞察 · 学习目标

1. 说出 installd 三件职责（目录属主/配额清退/Crate）与 PMS 的分工边界；2. 解释 split APK 在系统侧"一包多文件"与签名全一致要求；3. 对比三套备份通道的粒度与安全边界；4. 用 dumpsys/df 观测配额与 cache 清退痕迹

8.1 installd：配额与缓存清退官

`frameworks/native/cmds/installd/`：以 root 运行、binder 对接 PMS/系统（CacheTracker.cpp:36 起）。三件职责：**目录属主托管**（chown 到目标 uid/改 SELinux 标签——安装五步里 PMS 无权做的脏活）、**cache 配额与清退**（CacheTracker 按 app 聚合 cache 体积，磁盘紧张时由系统存储策略调用 rm_cache 级联清退，App 无感）、**Crate/私有目录**（scoped storage 的物理执行者）。配额底层是文件系统原生 quota 或 xattr 记账（按设备配置）——所以"cache 莫名变小"第一嫌疑就是它。

8.2 多 APK 与 Feature Delivery 的系统侧

split APK（base+config/split_*）在系统侧=一个 Package 多个 APK 文件（PackageApkInstaller 路径），签名须全一致；加载由 LoadedApk 的 split classloader 路由（衔接书 4 §1.4 注释①）。Play Feature Delivery 的 install-on-demand/conditional 只是**会话化安装**（书6 §3.1）在模块粒度上的复用——系统无需新机制，这是会话化设计的红利兑现。

8.3 备份恢复体系

三套通道并存：**Key/Value 备份**（BackupManager API → BMS（services/backup）→ 传输（Google/ OEM）逐键增量）；**全量备份**（adb backup / Auto Backup tar 流）；**设备到设备**（近场直传，13 主推）。关键机制：BMS 在安装会话后建"备份过期"标记，应用首次启动懒注册 Agent；恢复发生在安装流水线的"校验后、首启前"窗口——与 rollback 共用 staging 思想。安全边界：备份默认端到端加密密钥挂 Google/OEM 账号，工作资料独立。

源码锚点 · 本章锚点

frameworks/native/cmds/installd/{Installd,CacheTracker,CrateManager}.cpp
frameworks/base/services/backup/（BackupManagerService 家族）
core/java/android/app/backup/（BackupAgent/BackupDataInput 输出侧 API）
split 加载：android/app/LoadedApk（书4 §1.4 注释① 同源）

第九章 补全：APK 签名体系、Intent 匹配与安装后布局

设计洞察 · 学习目标

1. 对比签名 v1-v4 各代"防什么、短板在哪"并说出渠道包正确姿势；2. 走读 Intent 匹配四关并解释 CATEGORY_DEFAULT 潜规则；3. 说出 /data/app 安装后物理布局与 oat 产物的角色；4. 用 apksigner/pm query-activities 现场验证签名与匹配

9.1 签名四代（包宇宙的信任根）

方案	位置	防什么	短板
v1（JAR 签名）	ZIP 内逐文件 META-INF/MANIFEST.MF 条目	篡改任意文件	校验慢、不支持未压缩对齐完整性、渠道包注入漏洞源头
v2（APK Signing Block）	ZIP 中央目录前的整块	**整个 ZIP 字节级**篡改（含中央目录）	7.0 以下不认
v3（+密钥轮换谱系）	Signing Block 内附 proof-of-rotation 链	老密钥签新密钥的**升级链伪造**	链验证复杂
v4（fs-verity 树）	单独 .idsig 文件	运行时按页增量校验（配合 dm-verity 思想）	仅安装期用

安装时 `ApkSignatureVerifier`（core/java/android/util/apk/，公共校验器）与 `pm/PackageSignatures.java`（服务端指纹记账）按"最高可用方案"验证；升级一致性校验（书6 §3.2）比对的正是这套指纹。**平台签名**（platform key 签的 priv-app）等同系统身份（书6 §4.2 特权白名单的另一条腿）；密钥库（Keymint/TEE，后续册）管的是运行时密钥，与 APK 签名是两个世界。渠道包的正确姿势：v2+ 的 ZIP 尾部注释区或 ID-value 对（signing block 的额外 ID），别再碰 v1 注入。

9.2 Intent 匹配算法：queryIntentActivities 的另一半

第四册每一步"问地址"的算法本体在 **IntentResolver**（`services/core/java/com/android/server/IntentResolver.java:448` queryIntent → `:440` buildResolveList → `:428` queryIntentFromList）：

text 匹配四关（任一不过即淘汰）

```
① action：Intent.action 命中 filter 任一 action（隐式 Intent 必须 action 非空）
② category：Intent 每个 category 都被 filter 包含（CATEGORY_DEFAULT 潜规则：
目标 filter 必须显式声明 DEFAULT 才能被隐式启动——"少写一行清单"的经典坑）
③ data：scheme→host→port→path（pathPattern/pathPrefix 通配）逐级收窄，
+ mimeType（type 与 scheme=content/file 的联动）
④ 优先级与排序：系统 App > 三方；matched filter 数多者胜；
EXPORTED=false 或权限不符者在返回前剔除
```

有了这张四关表，"为什么我的隐式 Intent 起不来"排查顺序自然成立：action→category(默认)→data/type→exported/权限——每一关都能用 `pm query-activities` 或 dumpsys package 的 filter 段验证。

9.3 安装后的物理布局（/data/app）

```text /data/app/v~~<随机>/c~~<随机>/
├── base.apm→base.apk # 主体（split_*.apk 同级）
├── lib/<abi>/libxxx.so # 原生库（extractNativeLibs 决定是否解压）
└── oat/<abi>/base.odex|base.vdex # 编译产物（vdex=dex 缓存，odex=机器码，书6 §3.3）
```

`codePath/nativeLibraryDir` 记在 packages.xml（书6 §1.4 表）；首启的 LoadedApk 按它建类加载器与库路径。**v~~ 随机目录**防路径预测（书6 §3.1），oat 缺失则回落解释执行（书1 §8 三级策略的物理呈现）。

> **源码锚点 · 本章锚点**
> core/java/android/util/apk/ApkSignatureVerifier.java 校验 / pm/PackageSignatures.java 记账
services/core/.../IntentResolver.java:448/440/428
布局事实源：FileInstallArgs 落盘规则 + Settings.codePath

附录 A 观测实操

目标	命令	看点	
包全景	dumpsys package <pkg>	版本/签名/权限/用户态/InstallSource	
组件解析	pm query-activities --components ... 或 dumpsys package intents	倒排索引视角	
安装过程	pm install -r -t + logcat -s PackageManager	五步管线逐步日志	
失败码	INSTALL_FAILED_*	对照 §3.2 校验段	
权限态	appops get <pkg> / pm list permissions -g	三态+即时开关	
dex 状态	pm art dump（13）/ dumpsys package	dexopt	artd 视角
清账	pm clear / pm uninstall --user 0	用户态与全机删除差异	

附录 B 快问快答（16 条）

packages.xml 在哪？——/data/system/packages.xml（Settings 账本）。 Vending（Play）怎么静默装？——INSTALL_PACKAGES 系统签名+会话。 升级系统 App 的apk 放哪生效？——/data/app 覆盖（同签名高版本）。 scanDirTracedLI 扫几个分区？——framework、system(priv+normal)、overlay、vendor、product、system_ext、data（:248/:320/:325 及后续）。

priv-app 特权两个条件？——分区位置+privapp-permissions 白名单。 RRO 是什么？——运行时资源覆盖层（overlay 分区扫描）。 Computer 与 PMS 关系？——不可变查询快照，写后原子换引用。 installStage 在哪个类？——InstallParams.java:320（13 聚合进 InstallPackageHelper）。

安装到一半断电？——staging 目录被弃，账未动，旧版无损。 PACKAGE_ADDED 广播为何有序？——REPLACED→ADDED 链条语义敏感（备份/MA 依赖顺序）。 13 的 dexopt 谁执行？——ART APEX 的 artd（installd 退守存储）。 checkPermission 三层？——声明/授予/AppOps。

targetSdk 为什么影响权限？——M/23、Q/29、R/30 行为分水岭按其分叉。 多用户装一份包？——本体一份，PackageUserState per-user 记账。 拨号器为何出厂就有通讯录权限？——DefaultPermissionGrantPolicy 角色授予。 下一步读什么？——书7 面试题集全主题串联六册。

附录 C 自检题

- 默画安装五步，标注每步的"原子性保障点"。
- 扫描顺序为什么 data 最后？签名校验失败时哪一步拒绝？
- Computer 快照换引用的瞬间，读者拿到的是新是旧？为什么没有中间态？
- 权限三层各自由谁撤销？举一条命令对应每层。

附录 D 动手实验室（半天量）

实验一"账本考古"：root 后打开 /data/system/packages.xml，找一个你装的 App，核对 §1.4 十个字段；再 dumpsys package 同名包对照两本账的同一事实。

实验二"管线直播"：`logcat -s PackageManager` 开着，`pm install -r` 一个测试包，把日志按 §3.2 五步分段认领；故意换签名装第二遍，收割 `INSTALL_FAILED_UPDATE_INCOMPATIBLE`。

实验三"快照体验"：安装的同时连续 `dumpsys package <pkg> | grep version`——观察读到的是旧版本直到广播后跳新（Computer 语义现场版）。

实验四"预装演练"：在可写设备（模拟器）把一个普通 apk 放进 /system/priv-app（配白名单 xml），重启验证：特权生效了吗？构建期校验在城市里已跳过——运行时如何提示缺失白名单（logcat 的 PrivilegedPermission 检索）。

第十章 构造器与 systemReady：包宇宙的开机两幕

> **设计洞察 · 学习目标**
> 1. 把 PMS 构造器拆成"装配→对账扫描→收官"三段并说出每段的事件锚点与锁纪律；2. 区分构造器阶段与 systemReady 阶段各自的依赖前提与做不了的事；3. 走读 readLPw 的备份文件优先策略与 mIsUpgrade 的四项差异化处理；4. 用 boot_progress 事件序列量化开机扫描耗时并定位慢点

第一章讲的是扫描的"空间维度"（哪些目录、什么顺序），本章补上"时间维度"：PMS 从被 SystemServer 调用到宣告就绪，一共两幕——**构造器**（`SystemServer.startBootstrapServices` 里 `PackageManagerService.main`，书2 § 七的 Bootstrap 批）与 **systemReady**（`SystemServer.java:2712` 在 startOtherServices 尾段回调）。两幕的分工铁律：**构造器里能做的只有"自己 + 更早启动的依赖"**；AMS、存储、role 管理都还没起来，一切"要问别人"的事全部推迟到 systemReady。看懂这条分界线，开机扫描的很多代码就"各就各位"了。

10.1 链路位置：两幕各自的舞台

text 开机时间轴上的 PMS（对照第一章 bootscan 图）

```
t0 startBootstrapServices
├ Installer（installd 通道就绪，第八章 §8.1）
├ AMS（快速路径）→ startBootstrapServices 尾段：PackageManagerService.main
| └ 构造器第一幕：读账（readLPw）→ 扫描（initSystemApps/initNonSystemApps）
| → 写账（writeSettingsLPrTEMP）→ 发布 AIDL（ServiceManager.addService）
t1 startOtherServices
├ ……AMS/WMS/网络等大批服务启动（它们会立刻 queryIntent——走第二章快照）
├ :2712 mPackageManagerService.systemReady()
| └ 第二幕：对账收尾、默认权限重授、staged 会话恢复、后台 dexopt 注册
t2 phase 480/500/600 → BOOT_COMPLETED 广播
```

精读一句话：**构造器结束≠包宇宙可用**——AIDL 已发布（别的服务已经开始查询），但默认权限、staged 会话、AppsFilter 可见性都还没算完；这就是第二幕存在的理由。

10.2 构造器逐段：三段式与八个里程碑

java pm/PackageManagerService.java:1704-2255 构造器三段式（骨架+讲解注释）

```
public PackageManagerService(PackageManagerServiceInjector injector, boolean onlyCore,
boolean factoryTest, final String buildFingerprint, ...) {
// 讲解注释①【装配段】:1723 写入 BOOT_PROGRESS_PMS_START——bootstat 量开机的
// "包管理起点"，与 :1939 SCAN_START、:2056 SCAN_END、:2134 PMS_READY 四个
// 事件构成开机扫描的四把刻度尺（§10.4 用它排障）。
EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_START, SystemClock.uptimeMillis());
mLock = injector.getLock(); // 包宇宙大锁（内存账本）
mInstallLock = injector.getInstallLock(); // 安装/扫描锁（磁盘 I/O 侧，长事务）
// 讲解注释②：共享 uid 立项（:1764-1784）——android.uid.system/phone/nfc/bluetooth…
// 这是 packages.xml <shared-user> 标签的"合法名单"；不在名单里的 sharedUserId
// 协商直接失败（§3.5 表的 INSTALL_FAILED_NO_SHARED_USER 的上游）。
mSettings.addSharedUserLPw("android.uid.system", Process.SYSTEM_UID,
```

```
ApplicationInfo.FLAG_SYSTEM, ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
...
// 讲解注释③：辅助组件批量装配（:1837-1855）——13 职责拆分的"装配厂"落点：
// InstallPackageHelper/RemovePackageHelper/DeletePackageHelper/InitAppsHelper/
// DexOptHelper/SuspendPackageHelper/BroadcastHelper……PMS 从上帝类变门面。
mInstallPackageHelper = new InstallPackageHelper(this, mAppDataHelper);
mInitAppsHelper = new InitAppsHelper(this, mApexManager, mInstallPackageHelper,
mInjector.getSystemPartitions());
// 讲解注释④：第一台 Computer（:1862）——此后构造期的一切查询走这台快照；
// 构造器尾部 :2231 用终态再重建一次（第二章"改-建-换"的第一次实例化）。
mLiveComputer = createLiveComputer();
synchronized (mInstallLock) { synchronized (mLock) {
// 讲解注释⑤【对账段】内置共享库入库（android.test.runner 等由 SystemConfig
// 提供）+ SELinuxMMAC.readInstallPolicy() 读 seinfo 策略（书1 第十一章）。
// 讲解注释⑥：readLPw(:1906) 读 packages.xml——第一章 §1.4 的账本；返回值
// 即 mFirstBoot：无账本=首启，installd 需要 setFirstBoot（灰化编译缓存，
// 首启慢一截的元凶之一）。
mFirstBoot = !mSettings.readLPw(computer, mInjector.getUserManagerInternal()
.getUsers(true, false, false));
mPermissionManager.readLegacyPermissionsTEMP(mSettings.mPermissions);
// 讲解注释⑦：首启且非安全模式——把 /system 预编译的 vdex/odex 拷进 /data
//  （DexOptHelper.requestCopyPreoptedFiles:725）：预编译产物只读，而
// vdex 需要可写才能做运行时验证，拷贝=首启一次性买断后续启动速度。
if (!mOnlyCore && mFirstBoot) { DexOptHelper.requestCopyPreoptedFiles(); }
// 讲解注释⑧：mIsUpgrade(:1954)=指纹变了——OTA 语义总开关，驱动 §10.3 的
// 四项差异化处理（重授默认权限/清代码缓存/promote 系统权限/性能埋点）。
final VersionInfo ver = mSettings.getInternalVersion();
mIsUpgrade = !buildFingerprint.equals(ver.fingerprint);
EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_SYSTEM_SCAN_START, ...); // :1939
// 讲解注释⑨：两波扫描——先系统分区（含 APEX 派生分区，第十三章 §13.3），
// 后 /data/app；OverlayConfig 一并产出（RRO 世界线的入口，§9 RRO）。
mOverlayConfig = mInitAppsHelper.initSystemApps(packageParser, packageSettings,
userIds, startTime); // :1989
mInitAppsHelper.initNonSystemApps(packageParser, userIds, startTime); // :1991
EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_SCAN_END, ...); // :2056
// 讲解注释⑩：OTA 后重授默认权限——onStorageVolumeMounted 携带 mIsUpgrade，
// 最终落到第十二章 §12.5 的 restorePermissionState（"授予重演"）。
mPermissionManager.onStorageVolumeMounted(StorageManager.UUID_PRIVATE_INTERNAL,
mIsUpgrade);
// 讲解注释⑪：OTA 后清代码缓存但保留 profile（FLAG_CLEAR_CODE_CACHE_ONLY +
// FLAG_CLEAR_APP_DATA_KEEP_ART_PROFILES）——机器码作废、热点画像还在，
// 重编译走 speed-profile 而非全量，OTA 后首次使用不至于"全部解释执行"。
if (mIsUpgrade && !mOnlyCore) { ... mAppDataHelper.clearAppDataLIF(...); } // :2091
// 讲解注释⑫【收官段】先落盘再"点名"：writeSettingsLPrTEMP(:2132) 把扫描
// 结果原子写回 packages.xml（临时文件+rename，掉电不烂账，§3.2）。
writeSettingsLPrTEMP(); // :2132
EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_READY, ...); // :2134
// 讲解注释⑬：点名"不可缺席"的管家包——verifier/installer/uninstaller/
// PermissionController/sdk sandbox；getRequired*LPr 逐一按 intent 解析，
// 找不到或多于一个直接抛异常：系统拒绝带伤开机（§10.3 的设计洞察）。
mRequiredPermissionControllerPackage = getRequiredPermissionControllerLPr(computer);
// 讲解注释⑭：dex 使用统计建账（:2222 DexManager.load）——第三章 §3.3 策略
// 侧的另一半：安装时编译原因（install scenario）就由它裁定。
mDexManager.load(userPackages);
mLiveComputer = createLiveComputer(); // :2231 终态快照，第一幕落幕
```

```
} }
mInstaller.setWarnIfHeld(mLock); // :2251 之后 installd 调用若持包锁即打告警——
// 扫描期可以持锁，开机后绝不许
mServiceStartWithDelay = SystemClock.uptimeMillis() + (60 * 1000L);
}
```

逐段精读：①②的锁与共享 uid 都是"先立规矩后装人"——**mLock 管账、mInstallLock 管物**（书6 §3.6 的扫描走读即双锁齐持，注释⑨）。③值得单独记：13 把旧世界几千行的上帝方法拆成十几个 Helper，本章的 :1989/:1991 两行就是第一章整章内容的调用点——**教材里的整章，源码里往往只有一行**，这就是"聚合器"重构后的读码姿势。

10.3 mIsUpgrade 的四项差异化处理

OTA 后的第一次开机是特殊的：签名库可能新增（新版本权限）、机器码可能过期（ABI/编译器变了）、老权限模型需要迁移。源码用 `mIsUpgrade` 串起四件事：

处理	位置	为什么 OTA 后才做
重授默认权限	onStorageVolumeMounted(:2073) 携带 mIsUpgrade	新版本可能引入新权限，旧授予态需"授予重演"（第十二章 §12.5）
清代码缓存	mIsUpgrade 分支(:2091-2105)，保留 profile	预编译产物与新运行时不匹配，重编走 speed-profile
promote 系统权限	mPromoteSystemApps(:1962) 仅 pre-M 升级	老包 targetSdk<23 的 install 权限要升格为 runtime
性能埋点	BOOT_TIME_EVENT_DURATION__EVENT__OTA_PACKAGE_MANAGER_INIT_TIME(:2226)	OTA 首启的 PMS 耗时单独上报，regression 有哨兵

> **设计洞察 · 逐段精读：为什么"点名包"缺失要抛异常**
>
> verifier/installer/uninstaller 是安装链的必经站，PermissionController 是权限链的必经站（第四章 §4.3）。它们缺失时系统不会立即出错——会在第一次安装/第一次弹权限时半路搁浅，而且故障面遍历全机。与其让系统带伤运行到某个深夜炸出幽灵故障，不如在开机现场 fail-fast。
>
> getRequiredButNotReallyRequiredVerifierLPr 的 "NotReallyRequired" 自嘲恰好暴露设计张力：广播型 verifier 缺席可容忍（有超时兜底），PermissionController 缺席零容忍——**容忍度=下游是否有兜底**。

10.4 systemReady 逐段：第二幕的十件事

java pm/PackageManagerService.java:3993-4156 systemReady（骨架+讲解注释）

```
public void systemReady() {
PackageManagerServiceUtils.enforceSystemOrRoot(
```

```
"Only the system can claim the system is ready");
// 讲解注释①：F2fs 压缩块释放——增量安装（IncFs，第三章 §3.6 第③段）落盘的
// 压缩数据在此解压放气：开机早期 IO 拥挤，推迟到 systemReady 一次清账。
if (mReleaseOnSystemReady != null) {
for (...) { F2fsUtils.releaseCompressedBlocks(resolver, dstCodePath); }
}
mSystemReady = true;
// 讲解注释②：InstantApp 开关挂 ContentObserver——设置里关掉即时应用，
// 查询过滤即时生效（每 user 一个 mWebInstantAppsDisabled 位）。
// 讲解注释③：AppsFilter 可见性图就绪——"谁能看见谁"的邻接表（queryInterface
// 的过滤依据）在此建立；它依赖 UserManager 的完整用户列表，所以只能等第二幕。
mAppsFilter.onSystemReady(LocalServices.getService(PackageManagerInternal.class));
// 讲解注释④：运营商 App 延权——SIM 未载入前先禁用运营商预装，等 carrier
// privilege 就绪再放行；disableSkuSpecificApps 同理按硬件 SKU 收缩预装面。
CarrierAppUtils.disableCarrierAppsUntilPrivileged(...); // :4030
disableSkuSpecificApps();
synchronized (mLock) {
// 讲解注释⑤：mSettings.systemReady(:4046) 对比内存与 per-user 磁盘态，
// 返回"变了"的用户列表，逐个补写 package-restrictions.xml（第十二章
// §12.1 的三层账本之一）——开机对账的最后一笔。
ArrayList<Integer> changed = mSettings.systemReady(mComponentResolver);
for (int i = 0; i < changed.size(); i++) {
mSettings.writePackageRestrictionsLPr(changed.get(i));
}
}
mUserManager.systemReady();
storage.registerListener(mStorageEventHelper); // 拔插卷→可用性广播（第八章）
mInstallerService.systemReady(); // :4058 PackageInstallerService 就绪
mPackageDexOptimizer.systemReady();
// 讲解注释⑥：对账窗口——stale 用户清理与孤儿包清算（盘有账无/账有盘无），
// 第一章 §1.1 "两本账对账"原则在用户维度的延伸。
mUserManager.reconcileUsers(StorageManager.UUID_PRIVATE_INTERNAL);
mStorageEventHelper.reconcileApps(snapshotComputer(), StorageManager.UUID_PRIVATE_INTERNAL);
mPermissionManager.onSystemReady();
// 讲解注释⑦：权限指纹检查——runtime-permissions.xml 里记着上次的系统指纹，
// 过期（OTA）则对所有用户全量重授默认权限（DefaultPermissionGrantPolicy，
// 第四章 §4.2；第十二章 §12.4 的 300ms 延迟写在此大量触发）。
for (int userId : livingUsers) {
if (mSettings.isPermissionUpgradeNeeded(userId)) { grantPermissionsUserIds = ...; }
}
for (int userId : grantPermissionsUserIds) {
mLegacyPermissionManager.grantDefaultPermissions(userId); // :4082
}
...
mModuleInfoProvider.systemReady();
// 讲解注释⑧：最后一件事——恢复并应用 staged 会话（APEX/数据分区的"重启
// 生效"安装）。放在最后因为它可能安装包，而安装要求其他组件全部就绪；
// 链路详见第十三章 §13.4。
mInstallerService.restoreAndApplyStagedSessionIfNeeded(); // :4135
// 讲解注释⑨：后台 dexopt 双 Job 注册（JOB_POST_BOOT_UPDATE=801 开机后窗口、
// JOB_IDLE_OPTIMIZE=800 每日空闲窗）——第三章 §3.3 的策略层触发器。
mBackgroundDexOptService.systemReady();
// 讲解注释⑩：静态共享库的"过期缓存"清理闹钟（_pruneUnusedStaticSharedLibraries）。
schedulePruneUnusedStaticSharedLibraries(false);
}
```

设计洞察 · 逐段精读：为什么 staged 会话恢复必须是 systemReady 的最后一件事

restoreAndApplyStagedSessionIfNeeded（PackageInstallerService.java:326）会触发一次完整的安装管线（含 dexopt、默认权限授予、广播）。它依赖⑥的对账、⑦的重授、PackageDexOptimizer 就绪——顺序错了就是"用未初始化的子系统安装包"。源码注释原话 "Make sure this is the last component to be called"：**启动序即依赖图，最后一件的事往往最重要**。

10.5 观测：四把刻度尺排开机慢

- `adb logcat -b events | grep boot_progress`：四个事件（PMS_START:1723 / SYSTEM_SCAN_START:1939 / SCAN_END:2056 / PMS_READY:2134）的时间差即各段耗时——SCAN_START 到 SCAN_END 是纯扫描（第一章 §1.2 的目录循环），READY 前后是收官与点名。
- `adb logcat -s PackageManager`：扫描收尾会打 "Finished scanning system apps. Time: X ms, packageCount: N, cached: M"（InitAppsHelper.java:225）——`cached` 非 0 说明命中了 PackageCacher 的解析缓存（eng/userdebug 才开启，产品机恒为 0）。
- `adb shell dumpsys package | grep -A3 "ApexManager"`（第十三章 §13.2 的 apex 扫描规模）。
- 首启慢的另一元凶：讲解注释⑦的 requestCopyPreoptedFiles——用 `adb logcat | grep -i preopt` 与 `boot_progress_pms_start` 差值即可确认。

源码锚点 · 本章锚点

PackageManagerService.java:1704 构造器 / :1723/:1939/:2056/:2134 四个 boot_progress 事件
:1906 readLPw / :1927 requestCopyPreoptedFiles / :1954 mIsUpgrade / :2231 终态快照
systemReady :3993-4156 / mSettings.systemReady :4046 / restoreAndApplyStagedSessionIfNeeded :4135→PIS:326
SystemServer.java:2712 第二幕回调点 / InitAppsHelper.java:225 扫描耗时日志

第十一章 会话三阶段全链：commit、verify 与 install

设计洞察 · 学习目标

1. 把 commit 封印→stream 校验→verify 广播→install 四相串成一条消息序列并说出每段的 Handler；2. 说出 mSealed/mCommitted/stageDir 三个状态位在管线各阶段的角色；3. 解释 verifier 广播、integrity 校验与 enable-rollback 请求三路并行的会合方式；4. 用 logcat 与 perfetto 把一次真实安装按三阶段分段观测

第三章是安装管线的"总览图"，本章把 PackageInstallerSession 从 commit 到生效的**方法级**调用链走完。链路位置：第三章 §3.1 的 commit(:1638) 之后、§3.2 落盘五步之前——中间藏着 13 拆分出的三阶段状态机：**封印（seal）→ 流校验（stream validate）→ 验证（verify）→ 执行（install）**。整条链的线程模型先立此存照：

text 会话三阶段的消息序列（读码归纳，类均为 pm/ 下）

```
App/adb: session.commit(statusReceiver) [binder 线程]
└ MSG_ON_SESSION_SEALED → handleSessionSealed(:1675)
└ mCallback.onSessionSealedBlocking 持久化"已封印"位（PackageInstallerService）
└ MSG_STREAM_VALIDATE_AND_COMMIT(:1683)
└ handleStreamValidateAndCommit(:1688) [PMS handler 线程]
└ streamValidateAndCommit(:1828) 校验 staging 完整性 + CAS 置 mCommitted
└ MSG_INSTALL
└ handleInstall(:2121) → verify(:2146) [PMS handler 线程]
└ prepareInheritedFiles + parseApkAndExtractNativeLibraries
└ verifyNonStaged(:2303) → PackageSessionVerifier.verify(:96) [专用 handler]
└ VerificationParams.handleStartCopy(:165) [PMS handler]
└ integrity 校验请求 + verifier 广播 + enable-rollback 请求
└ 三路会合 → onVerificationComplete(:2413)
└ install(:2338) → makeInstallParams → InstallParams.installStage(:320)
└ INIT_COPY → PackageHandler.doHandleMessage(:84) → installPackagesLI 四相
└ POST_INSTALL → handlePackagePostInstall（广播链，第三章 §3.4）
```

11.1 阶段一：commit 与封印（事务的 BEGIN）

java pm/PackageInstallerSession.java:1638-1689 与 :1777（骨架+讲解注释）

```
public void commit(@NonNull IntentSender statusReceiver, boolean forTransfer) {
// 讲解注释①：多包会话（base+split 或跨包原子装）的子会话不允许单独 commit——
// 原子性的第一道防线：要么整批提交，要么一个都不进管线。
if (hasParentSessionId()) {
throw new IllegalStateException("Session " + sessionId
+ " is a child of multi-package session ...");
}
// 讲解注释②：markAsSealed(:1777) 做"最后校验+封印"：断言调用者是会话主人或
// root、statusReceiver 就位，然后置 mSealed=true——封印后 write/write 数据
// 接口全部拒绝，staging 目录内容从此只读（§3.1 讲解注释①的"零副作用"到此为止）。
if (!markAsSealed(statusReceiver, forTransfer)) { return; }
if (isMultiPackage()) {
synchronized (mLock) {
boolean sealFailed = false;
for (int i = mChildSessions.size() - 1; i >= 0; --i) {
// 讲解注释③：子会话逐一封印，任何一个失败也不提前返回——先把全部
// 状态走完再统一裁决，避免"半封印"的中间态。
if (!mChildSessions.valueAt(i).markAsSealed(null, forTransfer)) {
sealFailed = true;
}
}
if (sealFailed) { return; }
}
}
dispatchSessionSealed();
}
```

```
private void dispatchSessionSealed() {
// 讲解注释④：commit 的 binder 线程到此返回——后续全部转投 handler 串行执行。
// 为什么不在 binder 线程直接干？封印要 PackageInstallerService 持锁记账
//  （binder 线程不可长占），且三阶段必须串行，消息队列天然是状态机。
mHandler.obtainMessage(MSG_ON_SESSION_SEALED).sendToTarget();
}
private void handleSessionSealed() {
assertSealed("dispatchSessionSealed");
// 讲解注释⑤：onSessionSealedBlocking 把"已封印"落盘到 installer 的会话记录——
// 进程崩溃重启后，已封印会话不允许再写数据（防"封印后偷换内容"）。
mCallback.onSessionSealedBlocking(this);
dispatchStreamValidateAndCommit(); // → MSG_STREAM_VALIDATE_AND_COMMIT
}
```

设计洞察 · 设计洞察：为什么 commit 不做任何校验

commit 只封印、不校验——校验放到 streamValidateAndCommit 的原因是**提交意图与提交资格要分离**：调用方 commit 后可能立即崩溃，封印位已经落盘，数据不再可变；此时无论谁来推进管线，看到的都是同一份字节。事务隐喻：commit=BEGIN，streamValidateAndCommit=PREPARE，onVerificationComplete 后的 install=COMMIT——安装会话是教科书级的两阶段提交（2PC）实现。

11.2 阶段二：streamValidateAndCommit——数据完整性闸门

java pm/PackageInstallerSession.java:1828-1888（骨架+讲解注释）

```
private boolean streamValidateAndCommit() throws PackageManagerException {
try {
synchronized (mLock) {
if (mCommitted.get()) { return true; } // 幂等：重复推进直接放行
if (!params.isMultiPackage) {
// 讲解注释①：DataLoader 会话（增量/流式安装）在此挂上读通道——
// 后续校验按需拉数据页（IncFs 语义，第三章 §3.6 的 v4 签名配套）。
if (!prepareDataLoaderLocked()) { return false; }
// 讲解注释②：分流校验——APK 会话走 validateApkInstallLocked（:2677）：
// 逐个 ApkLite 轻解析，断言包名一致、split 名不重复、版本/签名各
// split 内部自洽——只读"目录头"，不做全量验签；APEX 会话则走
// validateApexInstallLocked（压缩 APEX 的 digest 核对）。
if (isApexSession()) { validateApexInstallLocked(); }
else { validateApkInstallLocked(); }
}
if (mDestroyed) {
throw new PackageManagerException(INSTALL_FAILED_INTERNAL_ERROR,
"Session destroyed");
}
// 讲解注释③：进度锁与活跃计数——客户端 close() 了会话句柄也不许此刻
// 被回收（mActiveCount+1），提交中的会话是"系统资产"不是"客户端资产"。
mActiveCount.incrementAndGet();
// 讲解注释④：CAS 置位 mCommitted——整个 PMS 里该会话的"提交"事实
// 只允许成立一次（compareAndSet 失败即内部错误）。这是 2PC 的
// PREPARE 成功标志，之后任何失败都要走 destroy() 清 staging。
if (!mCommitted.compareAndSet(false, true)) {
throw new PackageManagerException(INSTALL_FAILED_INTERNAL_ERROR,
"The mCommitted of session " + sessionId + " should be false");
}
```

```
committedMillis = System.currentTimeMillis();
}
return true;
} catch (PackageManagerException e) { throw e;
} catch (Throwable e) { throw new PackageManagerException(e); } // 异常统一化
}
```

逐段精读：②的"轻解析在校验期重做一次"不是浪费——staging 期写的数据与 commit 期的数据之间没有信任关系（可能是 DataLoader 分页补齐的），**每一阶段对上一阶段做防御性复核**，是长管线的自洽之道。validateApkInstallLocked(:2677) 里还有一个细节：`VerityUtils.hasFsverity(pkgInfo.applicationInfo.getBaseCodePath())` 探测旧版是否启用了 fs-verity，从而决定新装是否必须继承——**安全属性是会话间继承的契约**。

11.3 阶段三：verify——三路并行的外部协商

MSG_INSTALL 触发 `handleInstall(:2121)`：若有用户确认（Device Owner 静默限制等）先 `sendPendingUserActionIntentIfNeeded` 暂停等待；staged 会话转 `mStagedSession.verifySession()`（第十三章 §13.4），普通会话进 `verify(:2146)`——先 `prepareInheritedFiles`（MODE_INHERIT_EXISTING 的 split 更新要硬链接继承旧文件，能 link 不 copy）与 `parseApkAndExtractNativeLibraries`（extractNativeLibs=true 的包在此解 .so），然后进验证器：

java pm/PackageSessionVerifier.java:96-121 + VerificationParams.java:165-193（骨架+讲解注释）

```
public void verify(PackageInstallerSession session, Callback callback) { // :96
mHandler.post(() -> {
storeSession(session.mStagedSession);
// 讲解注释①：APEX 专项检查——checkApexUpdateAllowed（分级 APEX 只允许
// 指定更新方）、checkRebootlessApex（免重启 APEX 的白名单校验）。
checkApexUpdateAllowed(session); checkRebootlessApex(session);
verifyAPK(session, callback);
});
}
private void verifyAPK(PackageInstallerSession session, Callback callback) { // :122
final IPackageInstallObserver2 observer = new IPackageInstallObserver2.Stub() {
@Override public void onPackageInstalled(String basePackageName, int returnCode,
String msg, Bundle extras) {
// 讲解注释②：验证结果的会合点——staged 会话继续走 verifyStaged（预重启
// 验证），普通会话 setSessionReady 后回调管线继续。
if (session.isStaged() && returnCode == PackageManager.INSTALL_SUCCEEDED) {
verifyStaged(session.mStagedSession, callback); return;
}
...
}
};
final VerificationParams verifyingSession = makeVerificationParams(session, observer);
// 讲解注释③：verifyStage(:774) 把验证投回 PMS 主 handler——注意验证并没有
// 换进程，只是又排了一次队：让"校验发起"与"结果回执"异步解耦。
verifyingSession.verifyStage();
}
public void handleStartCopy() { // :165
PackageInfoLite pkgLite = PackageManagerServiceUtils.getMinimalPackageInfo(...);
// 讲解注释④：requiredInstalledVersionCode 校验——企业策略可要求"只能从
```

```
// 指定版本升级"，不满足直接 INSTALL_FAILED。
Pair<Integer, String> ret = mInstallPackageHelper.verifyReplacingVersionCode(
pkgLite, mRequiredInstalledVersionCode, mInstallFlags);
if (mRet != INSTALL_SUCCEEDED) { return; }
if (!mOriginInfo.mExisting) {
if ((mInstallFlags & PackageManager.INSTALL_APEX) == 0) {
// 讲解注释⑤：两路广播并行发出——sendIntegrityVerificationRequest(:251)
// 发给 integrity 校验器（Google Play Protect 的通道），sendPackageVer-
// ificationRequest(:335) 发给 ACTION_PACKAGE_NEEDS_VERIFICATION 广播
//  （mRequiredVerifierPackage 点名的那个 verifier，第十章 §10.3）。
// 双方各自回执，PackageVerificationState(:191) 计数会合。
sendApkVerificationRequest(pkgLite);
}
if ((mInstallFlags & PackageManager.INSTALL_ENABLE_ROLLBACK) != 0) {
// 讲解注释⑥：第三路——ACTION_PACKAGE_ENABLE_ROLLBACK 广播请 Rollback-
// Manager 冻结旧版（staging 快照指针，§3.4 回滚机制的物质准备），
// 默认超时 10s（DEFAULT_ENABLE_ROLLBACK_TIMEOUT_MILLIS）。
sendEnableRollbackRequest();
}
}
}
```

逐段精读：⑤⑥揭示了"为什么验证是独立阶段"——**验证的本质是向外部权威协商**（verifier/Integrity/RollbackManager），它们都是异步 binder/广播，必须有一个能"挂起等回执"的阶段。全部通过才进 `onVerificationComplete(:2413)`：staged 会话交给 StagingManager（第十三章），普通会话直接 `install(:2338)`。若验证期用户点了"拒绝"，`onSessionVerificationFailure` 把会话置 FAILED 并销毁 staging——旧版全程在线。

11.4 阶段四：installPackagesLI 四相（物化的机器房）

java pm/InstallPackageHelper.java:882-1060 installPackagesLI 四相（骨架+讲解注释）

```
private void installPackagesTracedLI(List<InstallRequest> requests) { // :882
try { Trace.traceBegin(TRACE_TAG_PACKAGE_MANAGER, "installPackages");
installPackagesLI(requests);
} finally { Trace.traceEnd(TRACE_TAG_PACKAGE_MANAGER); }
}
private void installPackagesLI(List<InstallRequest> requests) { // :911
// 讲解注释①：四相共享的中间产物容器——Prepare/Scan/Reconcile/Commit 每相
// 的输出都是下一相的输入，Map 以包名为键，天然支持多包原子安装。
final Map<String, ScanResult> preparedScans = new ArrayMap<>(requests.size());
...
try {
for (InstallRequest request : requests) {
// 讲解注释②【Prepare 相】:1070 preparePackageLI——全量解析 APK（此时
// 才做真解析，与 §11.2 的 ApkLite 轻解析呼应）、校验签名一致/降级拒绝/
// sharedUserId 冲突，产出 PrepareResult（解析参数+扫描标志）。
prepareResult = preparePackageLI(request.mArgs, request.mInstallResult);
// 讲解注释③【Scan 相】scanPackageTracedLI——第一章开机扫描的同一套
// 入库逻辑复用：建 PackageSetting、算 ABI、解析 seinfo、登记组件。
// 安装=一次"离线开机扫描"，这是两套代码最终收敛的深意。
final ScanResult result = scanPackageTracedLI(
prepareResult.mPackageToScan, prepareResult.mParseFlags,
```

```
prepareResult.mScanFlags, System.currentTimeMillis(),
request.mArgs.mUser, request.mArgs.mAbiOverride);
createdAppId.put(packageName, optimisticallyRegisterAppId(result));
}
synchronized (mPm.mLock) {
// 讲解注释④【Reconcile 相】reconcilePackages——多包一致性与系统现状
// 的全局裁决：共享库版本协商、KeySet 校验、权限重算准备（把
// reconciled 包与其权限快照对齐，第十二章 §12.5 的输入）。
reconciledPackages = ReconcilePackageUtils.reconcilePackages(
reconcileRequest, mSharedLibraries,
mPm.mSettings.getKeySetManagerService(), mPm.mSettings);
// 讲解注释⑤【Commit 相】commitPackagesLocked——唯一允许改系统状态的
// 相：mPackages/mSettings 入账、旧版 APK 移入删除队列、suspending/
// hiding 等派生状态更新。注释原文 "all predictable errors must be
// determined before this phase"——该失败的全在前面失败了。
commitRequest = new CommitRequest(reconciledPackages,
mPm.mUserManager.getUserIds());
commitPackagesLocked(commitRequest);
success = true;
}
// 讲解注释⑥：出锁后的收尾相——executePostCommitSteps(:2216)：prepareApp-
// DataPostCommit（installd 建目录/配额，第八章 §8.1）、prepareAppProfiles
// （安装期 profile 就位）、按 install scenario 裁定编译原因并跑 dexopt。
executePostCommitSteps(commitRequest);
} finally {
if (success) { ... } // 增量安装补发 verifier 回执（v4 root hash）
else { for (...) { ... 回滚 AppId、关 Freezer、失败码置 UNKNOWN } }
}
}
```

> **设计洞察 · 设计洞察：为什么"安装=一次开机扫描"**
>
> ②③揭示安装与开机扫描共用同一套 Prepare/Scan 代码——开机扫描是"批量安装已存在的包"，安装是"单包的开机扫描"。这条同构让升级语义天然正确（PackageSetting 字段一致）、让 priv-app 白名单/共享库/权限推断只维护一份逻辑。读懂 installPackagesLI 四相，回看第一章扫描就是"第四相前的循环体"——**两张图其实是同一张**。

11.5 收尾相与广播

`executePostCommitSteps(:2216)` 按包循环：`mAppDataHelper.prepareAppDataPostCommitLIF`（installd 建 CE/DE 目录并挂配额）、替换场景 `mDexManager.notifyPackageUpdated`、`mArtManagerService.prepareAppProfiles`（把安装会话携带的 profile 落位，供 speed-profile 使用）、最后按 `getCompilationReasonForInstallScenario` 算出的编译原因执行 dexopt（第三章 §3.3 的策略→执行分工）。dexopt 之后 `handlePackagePostInstall`（PackageHandler POST_INSTALL 消息，:100）发有序广播链——第三章 §3.4 的五环广播由这里启动。

观测三件套：`logcat -s PackageManager` 看 `init_copy / verification / installPackages` 分段；`perfetto` 搜 async trace `installStage/queueInstall/installPackages`（installStage :322-324 打的 async 桩）；`dumpsys package` 的 `installing` 段看在途会话。

源码锚点 · 本章锚点

PackageInstallerSession.java:1638 commit / :1671 dispatchSessionSealed / :1777 markAsSealed
:1828 streamValidateAndCommit / :2677 validateApkInstallLocked / :2121 handleInstall / :2146 verify
:2303 verifyNonStaged / :2413 onVerificationComplete / :2338 install / makeInstallParams
PackageSessionVerifier.java:96/122/187 verify/verifyAPK/verifyStaged
VerificationParams.java:165 handleStartCopy / :251 integrity 请求 / :335 verifier 广播 / :774 verifyStage
InstallParams.java:320 installStage / PackageHandler.java:84 INIT_COPY / InstallPackageHelper.java:911 installPackagesLI

第十二章 权限账本：持久化、恢复与升级重算

设计洞察 · 学习目标

1. 画出权限三层账本的物理布局（packages.xml/package-restrictions.xml/runtime-permissions.xml）与各自写时机；2. 走读 readLPw 的对账骨架与 readInstallPermissionsLPr 的字段语义；3. 走读 restorePermissionState 的授予重演与四类标志位；4. 解释"升级时授予态保留/新权限默认 deny/降 targetSdk 出 REVIEW"三条规则的源码出处

第四章讲的是权限的"检查时"世界（三层闸门），本章讲"关机后世界"：授予态怎么写盘、开机怎么恢复、应用升级时怎么重算。链路位置：开机侧挂在第十章 §10.2 的 readLPw 与 isPermissionUpgradeNeeded；安装/升级侧挂在第十一章 §11.4 Reconcile 相与 onPackageInstalled；授予动作侧是第四章 §4.3 PermissionController 回调的 grantRuntimePermission。**权限三层闸门是"读"，本章的三本账才是"存"**。

12.1 三本账的物理布局

账本	路径	记什么	写时机
packages.xml	/data/system/packages.xml	全机注册表：签名指纹/路径/install 权限（legacy 与 shared-user 的 perms 段）	writeSettingsLPrTEMP（构造器 :2132 与每次安装/变更）
package-restrictions.xml	/data/system/users/<id>/（Settings.java:1430）	per-user 安装态：installed/enabled/hidden/suspended/静态授予标志	writePackageRestrictionsLPr（变更与 systemReady 补写）

账本	路径	记什么	写时机
runtime-permissions.xml	/data/system/users/<id>/runtime-permissions.xml（:1437）	per-user 运行时授予态：granted/flags（含撤销来源标志）	RuntimePermissionPersistence 300ms 延迟批量写（§12.3）

三层账本的分法沿袭第四章 §4.4 的"按变更频率分治"：install 态几乎不变（跟全机账本走）、安装态偶变（per-user）、运行时授予高频小变（独立文件+延迟合并写）。**把不同写频率的状态放进不同文件，是磁盘磨损与锁竞争的双重优化。**

12.2 readLPw 对账骨架：一次开机的账本恢复

java pm/Settings.java:2938-3160 readLPw（骨架+讲解注释）

```
boolean readLPw(@NonNull Computer computer, @NonNull List<UserInfo> users) {
FileInputStream str = null;
if (mBackupSettingsFilename.exists()) {
// 讲解注释①：备份文件优先——writeLPr 每次写账先 copy 旧文件为 backup 再
// 原子 rename 新账（:2403 writeLPr）；重启读到 backup 说明主账本上次没
// 写完，宁可回滚一版也不能读半本。且此时若主文件也在，视为损坏直接删。
str = new FileInputStream(mBackupSettingsFilename);
PackageManagerService.reportSettingsProblem(Log.INFO,
"Need to read from backup settings file");
if (mSettingsFilename.exists()) { mSettingsFilename.delete(); }
}
...
if (str == null) {
if (!mSettingsFilename.exists()) {
// 讲解注释②：无账本=首启——VersionInfo 强制到当前版本，返回 false
// 即 mFirstBoot（第十章 §10.2 讲解注释⑥的来源）。
findOrCreateVersion(StorageManager.UUID_PRIVATE_INTERNAL).forceCurrent();
return false;
}
str = new FileInputStream(mSettingsFilename);
}
final TypedXmlPullParser parser = Xml.resolvePullParser(str);
int outerDepth = parser.getDepth();
while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
&& (type != XmlPullParser.END_TAG || parser.getDepth() > outerDepth)) {
String tagName = parser.getName();
// 讲解注释③：五类顶层标签各归其主（第一章 §1.4 字段表的"解析入口"）：
if (tagName.equals("package")) {
readPackageLPw(parser, users, originalFirstInstallTimes); // :3009
} else if (tagName.equals("permissions")) {
mPermissions.readPermissions(parser); // 全局权限定义（legacy）
} else if (tagName.equals("permission-trees")) {
mPermissions.readPermissionTrees(parser);
} else if (tagName.equals("shared-user")) {
readSharedUserLPw(parser, users); // :3015 shared-user 含 perms
} else if (tagName.equals("preferred-activities")) {
... // 默认偏好（第六章）
}
...
```

```
}
// 讲解注释④：对账第一刀——pendingPackages（shared-user 引用先于定义出现的
// 悬挂引用）二次尝试绑定；仍无主则按损坏数据丢弃并告警。
// 讲解注释⑤：per-user 副账本——对每个已知用户调 readPackageRestrictionsLPr
// (:1635) 读 package-restrictions.xml；OTA 迁移场景用 originalFirstInstallTimes
// 兜底 firstInstallTime 不丢失（源码注释的"migrated from per package"）。
for (UserInfo user : users) {
readPackageRestrictionsLPr(user.id, originalFirstInstallTimes); // :3147
}
// 讲解注释⑥：对账第二刀——遍历扫描结果，凡"账有物无"（packages.xml 记了但
// 分区里扫不到 APK）的 PackageSetting 标记清理；"升级未完成"的标灰。
// 第一章 §1.1 的"开机必对账"在此兑现。
return true;
}
```

逐段精读：①是整个 Settings 可靠性的锚——**先保旧、后写新、写完才换**的三代账本策略，与第三章 §3.2 的"临时文件+原子 rename"是同一条纪律的两半（读半边在 readLPw，写半边在 :2403 writeLPr）。任何"packages.xml 损坏导致应用丢失"的现场，先看 backup 文件是否存在、时间戳是否晚于主文件。

12.3 install 权限与 runtime 权限的两种记法

install 权限（normal/signature）全机一份，记在 packages.xml 的 <perms> 段；runtime 权限 per-user 一份，记在 runtime-permissions.xml。前者读回来即可用，后者还要经过 §12.5 的"重算"。先看 install 权限的解析（全文逐行）：

java pm/Settings.java:2207-2233 readInstallPermissionsLPr（逐行+讲解注释）

```
void readInstallPermissionsLPr(TypedXmlPullParser parser,
LegacyPermissionState permissionsState, List<UserInfo> users)
throws IOException, XmlPullParserException {
// 讲解注释①：被两处复用——packages.xml 顶层 <permissions>（:3604，老格式）
// 与 <shared-user> 内 <perms>（:4115，共享 uid 的聚合授予态）。
int outerDepth = parser.getDepth();
int type;
while ((type=parser.next()) != XmlPullParser.END_DOCUMENT
&& (type != XmlPullParser.END_TAG
|| parser.getDepth() > outerDepth)) {
if (type == XmlPullParser.END_TAG
|| type == XmlPullParser.TEXT) { continue; }
String tagName = parser.getName();
if (tagName.equals(TAG_ITEM)) { // 每条权限一个 <item name=.. granted=.. flags=..>
String name = parser.getAttributeValue(null, ATTR_NAME); // :320
final boolean granted = parser.getAttributeBoolean(null, ATTR_GRANTED, true);
final int flags = parser.getAttributeIntHex(null, ATTR_FLAGS, 0); // 十六进制
// 讲解注释②：一条记录复制给所有已知用户——install 权限按设计就是
// "装即授予、全用户一致"（源码 restorePermissionState 头注释的第一段），
// 所以旧格式里不需要 per-user 区分。
for (final UserInfo user : users) {
permissionsState.putPermissionState(
new PermissionState(name, false, granted, flags), user.id);
}
} else {
```

```
Slog.w(PackageManagerService.TAG, "Unknown element under <permissions>: "
+ parser.getName());
XmlUtils.skipCurrentTag(parser); // 未知标签跳过——向前兼容
}
}
}
```

runtime 权限的落盘由内部类 `RuntimePermissionPersistence`（Settings.java:5373）负责，三个设计点值得背下来：

- **300ms 延迟 + 30% 抖动的合并写**（WRITE_PERMISSIONS_DELAY_MILLIS=300、WRITE_PERMISSIONS_DELAY_JITTER=0.3，:5375-5376）：权限对话框连点几次只触发一次 IO；多用户同时授予时抖动错峰，避免垄断 PMS 锁（源码注释原话 "avoid monopolizing PMS lock when written for multiple users"）。
- **上限 2s**（MAX_WRITE_PERMISSIONS_DELAY_MILLIS）：持续授予也不能把数据压丢超过两秒——延迟合并与持久性之间画了硬边界。
- **指纹升级机制**（:5450 isPermissionUpgradeNeeded / :5456 updateRuntimePermissionsFingerprint）：文件里记着系统指纹，OTA 后不匹配则视为"需要重授"，挂到第十章 §10.4 讲解注释⑦的全量重授流程。

java pm/Settings.java:5373-5497 RuntimePermissionPersistence（骨架+讲解注释）

```
private static final class RuntimePermissionPersistence {
private static final long WRITE_PERMISSIONS_DELAY_MILLIS = 300; // :5375
private static final double WRITE_PERMISSIONS_DELAY_JITTER = 0.3; // :5376
private static final long MAX_WRITE_PERMISSIONS_DELAY_MILLIS = 2000; // :5378
// 讲解注释①：双 handler——mAsyncHandler（SystemBg）负责序列化 XML 的 CPU 活，
// mPersistenceHandler（BackgroundThread）负责真正的磁盘写：解析与 IO 分离，
// 任何一步都不占前台线程。
// 讲解注释②：mWriteScheduled per-user 位——同一用户的重复授予在延迟窗口内
// 合并（:scheduleWriteLocked 原子置位，写完清零）。
// 讲解注释③：writeStateForUserAsync(:5497) 对外入口——grantRuntimePermission/
// revokeRuntimePermission 落账后都调它；真正的 XML 由 com.android.permission.
// persistence.RuntimePermissionsPersistence 生成（runtime-permissions.xml）。
// 讲解注释④：isPermissionUpgradeNeeded(:5450) 读文件头指纹 vs 当前指纹——
// systemReady 的重授触发器（第十章 §10.4 讲解注释⑦）。
}
```

设计洞察 · 逐段精读：为什么 runtime 权限要搬出 packages.xml

packages.xml 是全机单文件，任何变更都要全量重写（几 MB 级）；而权限授予是"用户点一下弹窗"级频率。独立小文件+延迟合并写把每次 IO 从"全机账本重写"降为"单用户 KB 级追加"，同时把损坏半径限制在单用户。**写频率差异决定文件边界**——与第一章 §1.4 的 packages.xml 字段表对照，你会发现该表里没有"运行时授予"字段，不是遗漏，是分层。

12.4 授予重演：restorePermissionState 逐段

开机恢复与包升级都会走到同一个函数——`PermissionManagerServiceImpl.restorePermissionState`（permission/PermissionManagerServiceImpl.java:2496）。它不看"以前是什么"，而是**从**

manifest 出发重新推导一遍，再与账本对齐——这就是"授予重演"：

java permission/PermissionManagerServiceImpl.java:2496-2620（骨架+讲解注释）

```
private void restorePermissionState(@NonNull AndroidPackage pkg, boolean replace,
@Nullable String packageOfInterest, @Nullable PermissionCallback callback,
@UserIdInt int filterUserId) {
// 讲解注释①：头注释即规则书——install 权限装机即授；runtime 权限按 targetSdk
// 分流：pre-M 老包 dangerous 权限算 install 权限（兼容），M+ 才是运行时。
// 讲解注释②：第一轮循环——把"不需要问人就能给"的权限先分好类：
// 特权白名单命中（isPrivilegedPermissionAllowlisted）、签名可授（shouldGrant-
// SignaturePermission，含 shouldGrantPermissionByFlags 的 privileged-if-was-
// granted 特例：升级前有过就保留）、内部权限（shouldGrantInternalPermission）。
for (int i = 0; i < requestedPermissionsSize; i++) {
final Permission permission = mRegistry.getPermission(permissionName);
if (permission.isPrivileged() && checkPrivilegedPermissionAllowlist(pkg, ps, permission)) {
... }
if (permission.isSignature() && (shouldGrantPermissionBySignature(pkg, permission)
|| shouldGrantPermissionByProtectionFlags(...))) { ... }
...
}
// 讲解注释③：shared-user 聚合——同 uid 的兄弟包权限并集、targetSdk 取最小：
// uid 是权限的真正主体（第四章 §4.1 特例通道），账也要按 uid 算。
if (!ps.hasSharedUser()) { ... } else { ...遍历 getSharedUserPackages 取并集... }
synchronized (mLock) {
for (final int userId : userIds) {
final UidPermissionState uidState = userState.getOrCreateUidState(ps.getAppId());
if (uidState.isMissing()) {
// 讲解注释④：全新 uid 状态（新装/换 uid）——runtime 权限默认 deny，
// 但受限权限（hard/soft restricted）打 RESTRICTION_UPGRADE_EXEMPT
// 标志：未来系统升级把权限划入受限名单时，老授予不追溯撤销——
// "新政策不溯既往"原则的落点。
uidState.updatePermissionFlags(permission,
FLAG_PERMISSION_RESTRICTION_UPGRADE_EXEMPT, ...);
if (uidTargetSdkVersion < Build.VERSION_CODES.M) {
// 讲解注释⑤：pre-M 老包升上来的——打 REVIEW_REQUIRED +
// REVOKED_COMPAT：账面上"已授予"（老行为兼容），但标记"首次
// 使用需复核+兼容期自动回收"。targetSdk 23 的分水岭就在这两行。
uidState.updatePermissionFlags(permission,
PackageManager.FLAG_PERMISSION_REVIEW_REQUIRED
| PackageManager.FLAG_PERMISSION_REVOKED_COMPAT, ...);
uidState.grantPermission(permission);
}
}
if (replace) {
// 讲解注释⑥：升级场景（replace=true）——非 shared-user 先把原状态
// 快照进 origState 再 reset，然后逐条按②的分类重推授予；shared-user
// 走 revokeUnusedSharedUserPermissionsLocked：兄弟包升级后无人再
// 请求的运行时权限要回收（防"借壳保权"）。
origState = new UidPermissionState(uidState);
uidState.reset();
}
for (int i = 0; i < requestedPermissionsSize; i++) {
// 讲解注释⑦：主循环——对每条请求权限：未知权限跳过并 DEBUG 日志
//  ("Unknown permission"——换 ROM 后应用少权限的最常见日志）；
```

```
// runtime-only 权限对 pre-M 包直接 continue；
// implicit 权限（READ_MEDIA 系由 READ_EXTERNAL_STORAGE 拆分而来）
// 查 SplitPermissionInfo 自动继承旧授予——权限拆分时代的兼容桥。
...
}
// 讲解注释⑧：循环结束——授予/撤销有变化的用户追加进 updatedUserIds，
// 触发 §12.3 的延迟落盘；replace 且有 revoked 则额外通知
// PermissionController 刷新 UI 态。
}
}
}
```

调用来源对齐三处：开机——构造器 onStorageVolumeMounted(mIsUpgrade)（第十章 §10.2 讲解注释⑩）与 onSystemReady（:4389）；安装/升级——InstallPackageHelper 的 mPermissionManager.onPackageInstalled（:602/:2177/:3006/:3259 四个调用点）经 updatePermissions → :4028；用户动作——grantRuntimePermission/revokeRuntimePermission（:1313/:1489）。

> **设计洞察 · 设计洞察：为什么"重演"比"增量修补"可靠**
>
> 增量修补（记录 diff、回放 diff）要求 diff 存储本身可信且可迁移；重演只依赖两样强约束事实——manifest（随包自带）与权限定义（随系统自带）。OTA 换了权限模型、Manifest 改了请求、用户清了数据，重演都能收敛到一致态。代价是每次安装多一遍 O(权限数) 计算——相比正确性，这是白送的。**能用纯函数重建的状态，就不要发明增量协议。**

12.5 用户授予的前台动作：grantRuntimePermissionInternal

第四章 §4.3 说"授予动作仍是 PMS 单点记账"，本节给出那一行的实体（:1323）：

java permission/PermissionManagerServiceImpl.java:1323-1420（骨架+讲解注释）

```
private void grantRuntimePermissionInternal(String packageName, String permName,
boolean overridePolicy, int callingUid, final int userId, PermissionCallback callback) {
// 讲解注释①：调用方门槛——GRANT_RUNTIME_PERMISSIONS 是 signature|privileged
// 级权限（PermissionController/Shell/系统服务才有）；跨用户再过一道
// enforceCrossUserPermission（工作资料的隔离闸，第五章 §5.1）。
mContext.enforceCallingOrSelfPermission(
android.Manifest.permission.GRANT_RUNTIME_PERMISSIONS, "grantRuntimePermission");
enforceCrossUserPermission(callingUid, userId, true, true, "grantRuntimePermission");
// 讲解注释②：权限类型分流——isRuntime 正常走；isRole（拨号器这类"角色即权限"）
// 要求调用方持有 MANAGE_ROLES_FROM_CONTROLLER 权限；soft-restricted（如
// READ_EXTERNAL_STORAGE 在 R 上的软受限态）要过 SoftRestrictedPermissionPolicy
// 的 mayGrantPermission()——策略类可按 targetSdk 分设备版本定制。
isRolePermission = permission.isRole();
isSoftRestrictedPermission = permission.isSoftRestricted();
...
// 讲解注释③：受限权限的白名单闸——hard-restricted 权限不在白名单里直接拒绝；
// 白名单来源即 §12.4 onPackageInstalled 里 addAllowlistedRestrictedPermissions
// 传进来的 installer 白名单（:4985 onPackageInstalledInternal）。
// 讲解注释④：落账——uidState.grantPermission(permission) 改内存态，随后
// scheduleWrite → §12.3 的 300ms 延迟写 runtime-permissions.xml；
// onPermissionGranted callback 最终通知 PermissionController/AppOps 刷新。
```

```
}
```

观测现场：授予后立刻 `adb shell cat /data/system/users/0/runtime-permissions.xml | grep -A2 <perm>`，300ms 内文件未变、之后出现该条目——延迟写的行为可视化（userdebug）。

源码锚点 · 本章锚点

Settings.java:2938 readLPw / :2403 writeLPr / :1635 readPackageRestrictionsLPr / :1430/:1437 两份 per-user 文件
:2207 readInstallPermissionsLPr / :5373 RuntimePermissionPersistence / :5450 isPermissionUpgradeNeeded
PermissionManagerServiceImpl.java:2496 restorePermissionState / :1323 grantRuntimePermissionInternal
:4985 onPackageInstalledInternal / InstallPackageHelper.java:602/2177/3006/3259 onPackageInstalled 四调用点

第十三章 APEX：apexd、ApexManager 与包宇宙的双轨扫描

设计洞察 · 学习目标

1. 说清 APEX 的"原生激活（apexd）与 Java 扫描（ApexManager）"双轨分工与 ActiveApexInfo 四元组；2. 走读 scanApexPackagesInternalLocked 的并行解析与 apex-system-service 收集；3. 解释 APK-in-APEX 如何被派生成扫描分区（SCAN_AS_APK_IN_APEX）；4. 走通 staged APEX 的"预重启验证→checkpoint→重启激活"闭环

书1 §16.3 讲过 Mainline 模块的宏观图景，本册至此还没有正面解剖 APEX 在 PMS 内的存在方式。本章补齐：APEX 不是一个"安装包格式"，而是**双轨世界**——原生世界由 apexd（system/apex/apexd，init 早期启动）负责激活与挂载，Java 世界由 `ApexManager`（pm/ApexManager.java）作为唯一封装层向 PMS 翻译。链路位置：开机侧挂在第十章 §10.2 扫描的第一波（initSystemApps :1989 之前先做 APEX 解析）；安装侧挂在第十一章 §11.3 verifyStaged 的预重启验证。

13.1 双轨世界与 ActiveApexInfo

text APEX 的两条世界线（读码归纳）

```
原生轨：init 起 apexd → 校验/激活各模块 → bind-mount /apex/<name>@<ver> → /apex/<name>
（激活结果对整部设备可见，进程启动时的 linker namespace/classpath 由此定型）
Java 轨：PMS 构造器拿 mApexManager（注入器 :1451 ApexManager.getInstance()）
└ ApexManager 以 binder 调 IApexService（waitForApexService，:542-546）
├ getAllPackages() → 全部 APEX 模块（含未激活版本，:580）
└ getActivePackages() → 激活子集（缓存进 mActiveApexInfosCache，:538）
```

两轨的信息交汇点是 `ActiveApexInfo`（ApexManager.java:119-149），四个字段要背：

字段	语义	排障用途
apexModuleName	模块名（com.android.art 等）	`cmd apex/dumpsys` 对号
apexDirectory	激活后的挂载点（/apex/com.android.art）	PMS 扫描它（§13.3）
preInstalledApexPath	出厂预装位置（/system/apex/…apex 或 /data/apex/…）	映射回 system 分区（§13.3 注释③）
isFactory/isActive	出厂版？激活版？	判定"同模块双版本"合法性（§13.2 注释⑥）

> **设计洞察 · 设计洞察：为什么 PMS 不直接读 /apex 目录**
> 因为激活不是复制而是 mount——目录里的"当前版本"是 bind-mount 的产物，跨模块依赖（linker namespace）与校验（dm-verity/fs-verity）都是 apexd 的职责。PMS 通过 ApexManager 拿到的 ApexInfo 自带"哪些激活、哪些是出厂版"的裁决结果。**每个子系统只读自己有权裁决的事实的源头**——PMS 管 manifest 与签名，apexd 管挂载与激活，互不越界。

13.2 scanApexPackagesTraced：把 APEX 读成包

java pm/ApexManager.java:565-700 scanApexPackagesInternalLocked（骨架+讲解注释）

```
void scanApexPackagesTraced(@NonNull PackageParser2 packageParser,
@NonNull ExecutorService executorService) { // :565
Trace.traceBegin(TRACE_TAG_PACKAGE_MANAGER, "scanApexPackagesTraced");
synchronized (mLock) { scanApexPackagesInternalLocked(packageParser, executorService); }
Trace.traceEnd(TRACE_TAG_PACKAGE_MANAGER);
}
@GuardedBy("mLock")
private void scanApexPackagesInternalLocked(...) { // :578
mAllPackagesCache = new ArrayList<>();
// 讲解注释①：向 apexd 要全量清单——注意是 getAllPackages()（全部版本）而非
// getActivePackages()：PMS 需要感知"出厂版+更新版"并存的事实，才能做
// §13.4 的回滚与版本对账。binder 失败直接 RuntimeException——APEX 世界
// 没了，包宇宙无从建起（fail-fast，第十章 §10.3 同款哲学）。
allPkgs = waitForApexService().getAllPackages();
// 讲解注释②：每个 modulePath 交给 ParallelPackageParser 并行解析——与系统
// 分区扫描（第一章 §1.5 第③段）共用同一个解析器与线程池；APEX 也是 ZIP+
// AndroidManifest，PARSE_COLLECT_CERTIFICATES 让解析期就收签名。
for (ApexInfo ai : allPkgs) {
File apexFile = new File(ai.modulePath);
parallelPackageParser.submit(apexFile,
ParsingPackageUtils.PARSE_COLLECT_CERTIFICATES);
parsingApexInfo.put(apexFile, ai);
}
HashSet<String> activePackagesSet = new HashSet<>();
HashSet<String> factoryPackagesSet = new HashSet<>();
for (int i = 0; i < parsingApexInfo.size(); i++) {
ParseResult parseResult = parallelPackageParser.take();
ApexInfo ai = parsingApexInfo.get(parseResult.scanFile);
```

```
if (throwable == null) {
parseResult.parsedPackage.hideAsFinal();
// 讲解注释③：PackageInfoWithoutStateUtils.generate——把解析产物与
// ApexInfo 合成 PackageInfo（版本/签名/路径齐备），塞进
// mAllPackagesCache：此后 getPackageInfo/getAllPackages 走缓存，
// 不再触碰 apexd（第一幕内只扫一次）。
mAllPackagesCache.add(packageInfo);
for (ParsedApexSystemService service :
parseResult.parsedPackage.getApexSystemServices()) {
// 讲解注释④：apex-system-service 收集——模块可对外暴露系统级
// 服务实现（jar+名称+initOrder），但受 min_sdk/max_sdk 门禁
// （UnboundedSdkLevel 判定，Mainline 模块版本参差的兼容闸）。
// 激活版的服务加入 mApexSystemServices，SystemServer 后续按它加载。
if (minSdkVersion != null && !UnboundedSdkLevel.isAtLeast(minSdkVersion)) { continue
; }
if (maxSdkVersion != null && !UnboundedSdkLevel.isAtMost(maxSdkVersion)) { continue;
}
if (ai.isActive) {
// 重名服务直接抛异常——同一系统服务两个提供方是致命配置错误
mApexSystemServices.add(new ApexSystemServiceInfo(
service.getName(), service.getJarPath(), service.getInitOrder()));
}
}
// 讲解注释⑤：双向索引——包名→模块名、模块名→激活包名
// （mPackageNameToApexModuleName/mApexModuleNameToActivePackageName），
// §13.3 派生扫描分区与 §13.4 pruneCachedApksInApex 都查这两张表。
mPackageNameToApexModuleName.put(packageInfo.packageName, ai.moduleName);
if (ai.isActive) {
// 讲解注释⑥：一致性卫兵——两个激活包同名、两个激活包同模块、
// 两个出厂包同名（VNDK 例外）均 IllegalStateException。这三条
// 不变量是"版本图谱"可信的前提，违反即拒绝开机。
activePackagesSet.add(packageInfo.packageName);
mApexModuleNameToActivePackageName.put(ai.moduleName, packageInfo.packageName);
}
if (ai.isFactory) { factoryPackagesSet.add(packageInfo.packageName); }
} else if (throwable instanceof PackageManagerException) {
// 讲解注释⑦：安全模式例外——最小启动（mOnlyCore）下非 coreApp 的
// APEX 解析失败可跳过（INSTALL_PARSE_FAILED_ONLY_COREAPP_ALLOWED），
// 其余解析失败一律 IllegalStateException 拒绝开机。
if (e.error == PackageManager.INSTALL_PARSE_FAILED_ONLY_COREAPP_ALLOWED) { continue; }
throw new IllegalStateException("Unable to parse: " + ai.modulePath, throwable);
}
}
}
```

逐段精读：④是本章最容易被忽略的机制——**APEX 不只带代码，还带"系统服务实现"**（apex-system-service 清单标签），SystemServer 启动某些服务时会按 mApexSystemServices 选择模块内实现（initOrder 排加载序）。这解释了 Mainline 化的一个反直觉现象：改服务行为不一定要动 system_server 源码，升级模块即可。对照书1 §16.3 的模块列表，每个"可升级"背后都有这条通道。

13.3 APK-in-APEX：派生扫描分区

APEX 模块里还装着普通 APK（MediaProvider、PermissionController 等模块化系统组件——第四章 §4.3 说 PermissionController 是"独立 APK"，本体就住在 com.android.permission 模块里）。PMS 的做法不是特殊分支，而是**把每个激活 APEX 派生成一个"扫描分区"**：

java pm/InitAppsHelper.java:143-178（骨架+讲解注释）

```
private List<ScanPartition> getSystemScanPartitions() { // :143
final List<ScanPartition> scanPartitions = new ArrayList<>();
scanPartitions.addAll(mSystemPartitions);
// 讲解注释①：系统分区清单之后追加 APEX 派生分区——扫描顺序变为
// framework → system/vendor/product/system_ext → APEX → data（:105 的
// mDirsToScanAsSystem 即此函数的结果）。APEX 排在 data 之前：模块内 APK
// 仍是"系统应用"，必须先于 /data 同名包完成信任基准。
scanPartitions.addAll(getApexScanPartitions());
Slog.d(TAG, "Directories scanned as system partitions: " + scanPartitions);
return scanPartitions;
}
private List<ScanPartition> getApexScanPartitions() { // :151
final List<ScanPartition> scanPartitions = new ArrayList<>();
final List<ApexManager.ActiveApexInfo> activeApexInfos = mApexManager.getActiveApexInfos();
for (int i = 0; i < activeApexInfos.size(); i++) {
final ScanPartition scanPartition = resolveApexToScanPartition(activeApexInfos.get(i));
if (scanPartition != null) { scanPartitions.add(scanPartition); }
}
return scanPartitions;
}
private static @Nullable ScanPartition resolveApexToScanPartition( // :163
ApexManager.ActiveApexInfo apexInfo) {
for (int i = 0, size = SYSTEM_PARTITIONS.size(); i < size; i++) {
ScanPartition sp = SYSTEM_PARTITIONS.get(i);
// 讲解注释②：归属判定——APEX 的"出厂位置"（preInstalledApexPath，如
// /system/apex/com.android.permission.apex）落在哪个 system 分区下，
// 派生分区就继承该分区的信任级别。APEX 不是新信任域，是分区信任的
// 延伸——"特权由产地决定"在 APEX 时代依然成立。
if (apexInfo.preInstalledApexPath.getAbsolutePath().equals(
sp.getFolder().getAbsolutePath())
|| apexInfo.preInstalledApexPath.getAbsolutePath().startsWith(
sp.getFolder().getAbsolutePath() + File.separator)) {
// 讲解注释③：SCAN_AS_APK_IN_APEX(:378，PMS 位 23)——扫描时走
// "apex 内 APK"专用的路径/ABI/权限推断；激活版≠出厂版时
// SCAN_DROP_CACHE(:379) 强制丢弃解析缓存（PackageCacher 缓存的
// 是老版本产物，第一章 §1.5 的缓存优化在版本变化时必须失效）。
int flags = SCAN_AS_APK_IN_APEX;
if (apexInfo.activeApexChanged) { flags |= SCAN_DROP_CACHE; }
// 讲解注释④：扫描目录是激活挂载点 apexDirectory（/apex/<name>）——
// 版本路径的 @<ver> 细节被 mount 抹平，PMS 永远只看"当前激活"。
return new ScanPartition(apexInfo.apexDirectory, sp, flags);
}
}
return null;
}
```

此外 `initSystemApps(:190)` 在扫描前先 `mApexManager.scanApexPackagesTraced(...)`（§13.2），再取 `getApksInApex(:196-197)` 预建"包名→出厂路径"映射供 OverlayConfig 使用——RRO（第九章）判断 overlay 是否系统级时要问 APEX 归属。

> **设计洞察 · 设计洞察：派生分区而非特殊分支**
> PMS 没有为 APEX 写第二条扫描管线，而是把 apex 挂载点翻译成一个 ScanPartition 复用既有管线。收益有三：信任语义与分区一致（②）、扫描代码零分叉（第一章的 scanDirTracedLI 原样可用）、行为可观测（Slog 打出的 "Directories scanned as system partitions" 里 APEX 与分区同列）。**把新世界翻译成旧世界的类型，而不是给新世界写新代码**——框架演进的通用模式，与 13 把安装拆成 Helper 的"聚合器"（第十章 §10.2 注释③）异曲同工。

13.4 staged APEX：预重启验证与重启激活

APEX 安装（`pm install --apex` 或模块 OTA）不能即时生效——进程的 linker namespace 与 classpath 在启动时定型，**换 APEX 必须重启**。于是 APEX 会话走第十一章 §11.3 verifyAPK(:122) 里的 staged 分支：先做完整 APK 级验证，再进 `PackageSessionVerifier.verifyStaged(:187)` 的**预重启验证**：

java pm/PackageSessionVerifier.java:187-300 + StagingManager.java:501-524（骨架+讲解注释）

```
private void verifyStaged(StagingManager.StagedSession session, Callback callback) { // :187
mHandler.post(() -> {
try {
// 讲解注释①：活动会话互斥——同一时刻只允许一个 staged 会话集在途
//  （checkActiveSessions），避免两个会话对同一模块的激活竞争。
checkActiveSessions();
// 讲解注释②：回滚冲突检查——本会话若与已有 staged 回滚重叠即拒
//  （checkRollbacks）；多包会话还要逐一 checkOverlaps 防包集合交叠。
checkRollbacks(session);
...
dispatchVerifyApex(session, callback);
} catch (PackageManagerException e) { onVerificationFailure(session, callback, ...); }
});
}
private void verifyApex(StagingManager.StagedSession session) ... { // :262
// 讲解注释③：rollback 使能协商——INSTALL_ENABLE_ROLLBACK 时同步通知
// RollbackManager 冻结现状（rm.notifyStagedSession 返回 rollbackId）；
// APEX 回滚的 CE 数据快照（snapshotCeData，ApexManager:369）也在这条线里。
if ((session.sessionParams().installFlags & PackageManager.INSTALL_ENABLE_ROLLBACK) != 0) {
rollbackId = rm.notifyStagedSession(session.sessionId());
}
final boolean hasApex = session.containsApexSession();
if (hasApex) {
// 讲解注释④：提交给 apexd——submitSessionToApexService 最终调
// ApexManager.submitStagedSession(:843)：waitForApexService().
// submitStagedSession(params, apexInfoList)，apexd 在原生侧完成
// 签名/依赖校验并返回将激活的 ApexInfoList；任一侧失败抛
// INSTALL_FAILED_VERIFICATION_FAILURE（"apexd verification failed"）。
final List<PackageInfo> apexPackages = submitSessionToApexService(session, rollbackId);
// 讲解注释⑤：Java 侧再验签——对 apexd 返回的每个包跑 validateApex-
```

```
// Signature（与平台/出厂签名比对，v1-v4 校验器复用，第九章 §9.1）。
// 双侧校验：apexd 管"挂载安全"，PMS 管"包宇宙账本一致"。
for (int i = 0, size = apexPackages.size(); i < size; i++) {
validateApexSignature(apexPackages.get(i));
}
packageManagerInternal.pruneCachedApksInApex(apexPackages); // 缓存失效
}
}
void commitSession(@NonNull StagedSession session) { // StagingManager:501
createSession(session); // 登记到 mStagedSessions（崩溃后可恢复）
handleCommittedSession(session); // 已 ready 的 APEX 会话通知观察者
}
```

验证全通过后会话置 READY，`endVerification` 打开文件系统 **checkpoint**（用户数据快照，fs-rollback 模式的保险）。重启后两条线汇合：apexd 在原生侧激活新版本并 bind-mount；Java 侧则由第十章 §10.4 讲解注释⑧的最后一棒 `restoreAndApplyStagedSessionIfNeeded`（PackageInstallerService.java:326）接管——确认激活成功则把 staged 会话转正并补跑 APK 部分的完整安装（模块内 APK 更新、广播链）；激活失败（apexd revert）则标记会话失败并清理 staging。观测命令：

- `pm get-stagedsessions` / `pm list staged-sessions`（PackageManagerShellCommand.java:333/3937）——看在途/已就绪会话与 sessionReady 状态。
- `dumpsys package` 的 ApexManager 段（ApexManager.java:1258 dump 实现）——激活/出厂版本对照。
- `logcat -s StagingManager PackageSessionVerifier apexd`——预重启验证与激活全链日志。

设计洞察 · 设计洞察：为什么 APEX 安装要"重启"而 APK 不用

APK 的世界（四大组件、资源、dex）在 PMS/AMS 的运行时里有热更新通道（广播、组件重注册、ART 校验）。而 APEX 改写的是**进程世界的地基**：linker namespace、classpath、SELinux 属性、模块内 so 的版本绑定——这些都是"进程创建时刻"的快照。与其发明一套不可靠的热切换，不如复用开机这台"全量事务"：checkpoint 保数据，apexd 保挂载，PMS 保账本。**当状态不可热切换时，把生效点推迟到下一次全量重建（开机）**——与 A/B OTA（无缝升级）是同一设计直觉。

源码锚点 · 本章锚点

ApexManager.java:119-149 ActiveApexInfo / :565 scanApexPackagesTraced / :578 scanApexPackagesInternalLocked
:843 submitStagedSession / :1258 dump / PackageManagerService.java:378/379 SCAN_AS_APK_IN_APEX/SCAN_DROP_CACHE
InitAppsHelper.java:143 getSystemScanPartitions / :151 getApexScanPartitions / :163 resolveApexToScanPartition
PackageSessionVerifier.java:187 verifyStaged / :262 verifyApex / StagingManager.java:501 commitSession
PackageInstallerService.java:326 restoreAndApplyStagedSessionIfNeeded / PackageManagerShellCommand.java:333 get-stagedsessions

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. **PMS 构造器里"两把锁"（mLock 与 mInstallLock）各管什么？为什么扫描期要双锁齐持?**

参考答案要点：mLock 管内存账本（mPackages/mSettings 的组件与包状态），mInstallLock 管磁盘侧长事务（解析、落盘、dexopt 等慢 I/O）；扫描既改账又读盘，故构造器与 initSystemApps 双锁齐持；开机后 installd 调用持 mLock 会被 setWarnIfHeld 告警（PackageManagerService.java:2251）——两锁隔离慢 I/O 与热查询。

2. **开机扫描的四个 boot_progress 事件是什么？各自的源码位置?**

参考答案要点：BOOT_PROGRESS_PMS_START:1723（构造器入口）、SYSTEM_SCAN_START:1939（扫描开始）、SCAN_END:2056（扫描结束）、PMS_READY:2134（收官落盘完成）；SCAN_START 到 SCAN_END 的差值即纯扫描耗时，可用 `logcat -b events` 直接量（第十章 §10.5）。

3. **readLPw 为什么优先读 backup 文件？主文件与 backup 同时存在时怎么处理?**

参考答案要点：writeLPr 每次写账先把旧账拷为 backup 再原子 rename 新账；读到 backup 说明主账上次没写完，宁回滚一版不读半本；此时主文件视为损坏直接 delete（Settings.java:2938-2951）——三代账本纪律的读侧。

4. **会话从 commit 到生效要经过哪几条 handler 消息？各消息的处理方法是什么?**

参考答案要点：MSG_ON_SESSION_SEALED→handleSessionSealed(:1675)（封印落盘）→MSG_STREAM_VALIDATE_AND_COMMIT→handleStreamValidateAndCommit(:1688)（streamValidateAndCommit:1828 校验+CAS 置 mCommitted）→MSG_INSTALL→handleInstall(:2121)→verify(:2146)→verifyNonStaged→验证器

→onVerificationComplete→install→installStage（第十一章 §11.1）。

5. **mCommitted 的 CAS（compareAndSet）保护的是什么事实？失败意味着什么？**

参考答案要点：保护"该会话的提交事实只成立一次"（2PC 的 PREPARE 标志）；CAS 失败说明内部状态机被破坏（重复推进），抛 INSTALL_FAILED_INTERNAL_ERROR（PackageInstallerSession.java:1867-1872）——用原子变量替代布尔检查的经典幂等闸。

6. **权限三本账分别记什么、放在哪、何时写？**

参考答案要点：packages.xml（/data/system/）记全机注册表与 legacy install 权限，writeSettingsLPrTEMP 全量原子写；package-restrictions.xml（/data/system/users/<id>/，Settings.java:1430）记 per-user 安装态，writePackageRestrictionsLPr 变更即写；runtime-permissions.xml（:1437）记 per-user 运行时授予，RuntimePermissionPersistence 300ms 延迟合并写（第十二章 §12.1）。

7. **restorePermissionState 为什么叫"授予重演"？它的两个输入是什么？**

参考答案要点：它不回放历史 diff，而是从 manifest 的 requestedPermissions 与系统权限定义（mRegistry）出发重新推导一遍授予态，再与账本对齐；两输入都是"随载体自带"的强约束事实，OTA/升级/清数据后都能收敛（PermissionManagerServiceImpl.java:2496，第十二章 §12.4）。

8. **FLAG_PERMISSION_RESTRICTION_UPGRADE_EXEMPT 与 REVOKED_COMPAT 各防什么？**

参考答案要点：UPGRADE_EXEMPT 给新 uid 的受限权限打豁免——未来系统把权限划入受限名单时不追溯撤销老授予（新政策不溯既往）；REVOKED_COMPAT+REVIEW_REQUIRED 给 pre-M 老包——账面保留授予（兼容）但标记"首次使用复核+兼容期回收"（:2620-2628，targetSdk 23 分水岭）。

9. **ApexManager 为什么要向 apexd 要 getAllPackages() 而不是只拿激活包？**

参考答案要点：PMS 要感知"出厂版+更新版并存"的事实，才能做 staged 会话对账、回滚版本判断与"两个出厂包同名"卫兵；getActivePackages 缓存的只是激活子集（ApexManager.java:538/:580，第十三章 §13.2 注释①）。

10. **SCAN_AS_APK_IN_APEX 与 SCAN_DROP_CACHE 分别在什么条件下打上？**

参考答案要点：resolveApexToScanPartition（InitAppsHelper.java:163-178）把 preInstalledApexPath 归属到 system 分区后打 SCAN_AS_APK_IN_APEX（PMS:378，位 23）；activeApexChanged（激活版≠出厂版）时再打 SCAN_DROP_CACHE（PMS:379）强制丢弃 PackageCacher 的老版本解析缓存。

第二级：分析题（检验机制理解）

11. **追踪调用链：一次 `adb install` 从 PackageInstaller.commit 到有序广播链，标出每一步的方法锚点与所在线程。**

参考答案要点：commit:1638（binder 线程）→封印+MSG_ON_SESSION_SEALED（handler）→streamValidateAndCommit:1828→MSG_INSTALL→handleInstall:2121→verify:2146→verifyNonStaged:2303→PackageSessionVerifier.verify:96（verifier handler）→VerificationParams.handleStartCopy:165（PMS handler，integrity+verifier+rollback 三路广播）→onVerificationComplete:2413→install:2338→InstallParams.installStage:320→INIT_COPY（PackageHandler:84）→installPackagesLI:911 四相→executePostCommitSteps:2216→POST_INSTALL（PackageHandler:100）→handlePackagePostInstall 广播链（第十一章 §11.1 开头序列图）。

12. **失败推演：streamValidateAndCommit 通过后、verify 开始前进程崩溃，重启后会话与 staging 目录各是什么命运？**

参考答案要点：封印位与提交位已持久化（onSessionSealedBlocking 落盘），staging 内容不可变；重启后 restoreAndApplyStagedSessionIfNeeded（若 staged）恢复推进，非 staged 会话则由 PackageInstallerService 读会话元数据重建——mSealed 会话永远不允许再写数据，弃用需显式 abandon 清 staging；旧版本全程在线（第十一章 §11.1/§11.2，事务边界不依赖进程存活）。

13. **失败推演：OTA 后 PermissionController 包名解析失败（config_permissionControllerPackageName 指向不存在的包），开机走到哪一步停下？**

参考答案要点：构造器收官段的 getRequiredPermissionControllerLPr(:2167) 找不到包直接抛异常，PMS.main 失败，SystemServer Bootstrap 批崩溃进入 reboot loop——不会"带伤"进桌面；对比 verifier 缺席只打 "There should probably be a verifier" 警告（getRequiredButNotReallyRequiredVerifierLPr 有超时兜底）——容忍度取决于下游有无兜底（第十章 §10.3）。

14. **对比分析：installPackagesLI 四相中哪几相在 mInstallLock、哪几相在 mLock？为什么 Reconcile 和 Commit 要进 mLock 而 Prepare/Scan 不必？**

参考答案要点：Prepare:1070 与 Scan:947 只读解析产物与分区事实（mInstallLock 挡并发扫描），Reconcile（reconcilePackages 对 mPackages/mSettings/KeySet 全局裁决）与 Commit:commitPackagesLocked（唯一改系统状态的相）都持 mPm.mLock——**慢的放 InstallLock、改共享账本的放 mLock**，与第十章两锁分工一脉相承（InstallPackageHelper.java:911-1040）。

15. **跨册串联：本章哪些机制依赖 binder 的进程死亡语义或异步回执？各用了什么特性？**

参考答案要点：①会话验证的 IPackageInstallObserver2Stub 回执（binder 双向调用，书3 §四）；②verify/verifyStaged 的 mHandler.post 串行化——binder 线程只收请求不干活；③ApexManager 对 IApexService 的 binder 失败转 RuntimeException（远程服务死亡的 fail-fast）；④runtime 权限授予回调 PermissionCallback 通知 PermissionController（跨进程策略刷新）（第十一章/第十二章/第十三章）。

16. **失败推演：某预装 priv-app 升级后新增一个特权权限但没更新 privapp-permissions 白名单，开机与运行时分别发生什么？**

参考答案要点：userdebug/eng 版运行时 logcat 打 PrivilegedPermission 拒绝告警并按未授予处理（构建期 host_configverification 在产品机已前移到编译时报错）；restorePermissionState 的 checkPrivilegedPermissionAllowlist(:2539) 不命中，权限保持默认 deny——特权是"分区+白名单"双条件（第四章 §4.2、第十二章 §12.4 注释②）。

17. **对比分析：APEX 派生扫描分区与普通 system 分区扫描的三个差异点。**

参考答案要点：①目录来源——apexDirectory（激活挂载点）vs 分区目录；②标志——SCAN_AS_APK_IN_APEX 让路径/ABI/权限推断走 apex 语义，SCAN_DROP_CACHE 在版本变化时弃缓存；③前置依赖——必须先 scanApexPackagesTraced 拿到 ActiveApexInfo 才能派生（initSystemApps :190 先于 scanSystemDirs），普通分区无此依赖（第十三章 §13.3）。

第三级：设计题（检验架构能力）

18. **为 PMS 设计"三层失效半径"故障注入矩阵：针对文件级（账本）、文件系统级（checkpoint）、内存级（快照）各设计一个注入点与期望恢复行为，并说明三层半径的差异。**

参考答案要点：文件级——writeLPr 写账中途 kill，期望 readLPw 回退 backup 或读到完整旧账（Settings.java:2938-2951）；文件系统级——staged APEX 激活失败，期望 checkpoint 回滚用户数据且会话标 FAILED（第十三章 §13.4）；内存级——安装中并发查询，期望 Computer 快照恒为旧或新、无中间态（第二章）。失效半径依次为"单文件→整卷用户数据→进程内视图"，注入矩阵按半径从小到大排即冒烟到压力（第十章 §10.2、第十二章 §12.2、第十三章 §13.4）。

19. **为 PMS 设计一个"开机扫描耗时预算"机制：预算超限时自动降级。给出可降级的项、判定信号与风险。**

参考答案要点：信号用四个 boot_progress 事件差值（第十章 §10.5）与 InitAppsHelper 的 packageCount/timePerPackage 日志；可降级项按风险排序——①并行解析线程数（InitAppsHelper 的 executorService，纯吞吐）②PackageCacher 缓存粒度（牺牲内存换解析）③推迟非 coreApp 的 /data 扫描（§1.1 的 getAppInstallDir 分量，BOOT_COMPLETED 前补扫，风险是启动期第三方组件不可见）④ApkLite 先行的范围扩大（已有机制，第一章 §1.3）。不可降级项：framework/priv-app 扫描与两本账对账——信任链与自愈的根基（第一章 §1.2/§1.1）。

20. **如果要把 runtime-permissions.xml 的 300ms 延迟写改成"事务日志（WAL）+后台合并"，你会怎么设计？与现有机制对比得失。**

参考答案要点：每次授予 append 一条 (userId, pkg, perm, granted, flags, seq) 日志并 fsync，后台合并进主文件——持久性从 2s 上限提升到"写即落盘"，崩溃后重放 seq 即可恢复；代价是授予路径多一次 sync IO（高频授予的延迟敏感点）与重放代码的复杂度。现有机制的优点恰是"合并+抖动"把 IO 压到最低且损坏半径小（RuntimePermissionPersistence:5375-5378）；折中方案：保持延迟写、把 MAX_WRITE_PERMISSIONS_DELAY_MILLIS 作为可配置项并加"授予后 5s 强制刷"——大多数面试者应答出"先问要不要 WAL，再谈怎么做"（第十二章 §12.3）。

术语表（按拼音排序）

术语	解释
ActiveApexInfo	ApexManager 输出的激活 APEX 四元组：apexModuleName/apexDirectory（挂载点）/preInstalledApexPath（出厂路径）/isFactory+isActive（ApexManager.java:119-149），是 APEX 双轨世界的信息交汇点
ApexManager	PMS 与 apexd 之间的唯一 Java 封装层：scanApexPackagesTraced 把 APEX 读成包，submitStagedSession 对接预重启验证（pm/ApexManager.java，第十三章）
apex-system-service	APEX 模块对外暴露的系统级服务实现（名称+jar+initOrder），受 min/max_sdk 门禁；SystemServer 按 mApexSystemServices 加载模块内实现（ApexManager.java:625 起）
ApkLite 轻解析	只读 ZIP 中央目录与 manifest 头的轻量解析（包名/版本/签名/split 名），用于开机扫描初判与会话校验；全量解析推迟到 Prepare 相（第一章 §1.3、第十一章 §11.2）
backup settings 文件	packages.xml 的上一代账本：writeLPr 先拷旧账再原子 rename 新账，readLPw 见 backup 即优先读之并删除可疑主文件（Settings.java:2938-2951，第十二章 §12.2）
boot_progress 事件	开机扫描四把刻度尺：PMS_START:1723/SYSTEM_SCAN_START:1939/SCAN_END:2056/PMS_READY:2134，logcat -b events 可量各段耗时（第十章 §10.5）
CAS 提交位（mCommitted）	会话提交事实的原子闸：streamValidateAndCommit 末尾 compareAndSet(false,true)，失败即内部错误——2PC 的 PREPARE 标志（PackageInstallerSession.java:1867，第十一章 §11.2）
checkpoint	文件系统用户数据快照：staged APEX 验证通过后开启，重启激活失败可整体回滚用户数据（fs-rollback 模式），与 apexd 的模块回滚配合（第十三章 §13.4）
Computer 快照	13 起的不可变查询对象：写路径"改-建-换"原子换引用，读路径无锁；构造器 :1862/:2231 两次实例化（第二章，PMS 核心改造）
DataLoader	流式/增量安装的数据供给通道：会话校验期经 prepareDataLoaderLocked 挂载读通道，按需拉数据页，配合 v4 签名的 root hash（PackageInstallerSession.java:1841，第三章 §3.6）
markAsSealed	会话封印动作：断言调用者身份、置 mSealed、封印后写接口全部拒绝且封印位落盘——staging 内容从此只读（PackageInstallerSession.java:1777，第十一章 §11.1）

术语	解释
package-restrictions.xml	per-user 安装态账本（installed/enabled/hidden/suspended），路径 /data/system/users/<id>/（Settings.java:1430），systemReady 时对"changed 用户"补写（PackageManagerService.java:4046）
packages.xml	全机包注册表：签名指纹/路径/版本/install 权限/shared-user；Settings 账本的物理形态，构造器 readLPw 读、writeLPr 原子写（第一章 §1.4）
ParallelPackageParser	扫描共用的并行解析器：submit/take 队列模型，系统分区与 APEX 模块共用同一线程池与解析器（ApexManager.java:596-604，第一章 §1.5）
PackageCacher	解析结果缓存：eng/userdebug 把 ParsingPackage 序列化缓存加速开机；活跃 APEX 版本变化时 SCAN_DROP_CACHE 强制失效（InitAppsHelper.java:172）
restorePermissionState	授予重演：从 manifest+权限定义重新推导授予态并与账本对齐，开机（onStorageVolumeMounted/onSystemReady）与安装升级（onPackageInstalled）共用（PermissionManagerServiceImpl.java:2496，第十二章 §12.4）
runtime-permissions.xml	per-user 运行时授予账本：granted/flags（含 REVOKED_COMPAT 等标志），300ms 延迟合并写（Settings.java:1437/:5375，第十二章 §12.3）
RuntimePermissionPersistence	运行时权限落盘器：300ms+30% 抖动合并写、2s 上限、指纹升级检查 isPermissionUpgradeNeeded（Settings.java:5373-5497）
SCAN_AS_APK_IN_APEX	扫描位 23（PMS:378）：APEX 激活挂载点派生成扫描分区时打上，走"apex 内 APK"专用路径/权限推断（第十三章 §13.3）
streamValidateAndCommit	会话三阶段的数据完整性闸门：轻解析复核、CAS 置提交位、mActiveCount 保活——PREPARE 相本体（PackageInstallerSession.java:1828，第十一章 §11.2）
派生扫描分区	把 APEX 挂载点翻译成 ScanPartition 复用既有扫描管线的设计：信任级别继承出厂分区归属，扫描代码零分叉（InitAppsHelper.java:163-178，第十三章 §13.3）
两本账对账	内存 mPackages（账）与盘上 APK 文件（物）开机必核对：物亡删账、账亡弃物、升级未完成标灰——数据库思维的系统级自愈（第一章 §1.1）
特权双条件	priv-app 特权=分区位置（priv-app 目录）+privapp-permissions 白名单，缺一构建期（产品机）即拒；运行时按 checkPrivilegedPermissionAllowlist 裁定（第四章 §4.2）
授予重演	同 restorePermissionState：不存 diff、只存事实，可从两个强约束输入收敛——"能用纯函数重建的状态就不发明增量协议"（第十二章 §12.4）

术语	解释
写延迟合并	RuntimePermissionPersistence 的 IO 策略：300ms 延迟 +30% 抖动错峰合并多用户写入、上限 2s——把高频小写压成低频整写（第十二章 §12.3）

排障手册：PMS 与包管理问题定位

定位总原则：先定层（账本/管线/权限/APEX），再归因，后修复。本册域内的证据源有四：dumpsys package（两本账的读侧视图）、pm 系列命令（会话/路径/权限）、logcat 的 PackageManager/PermissionManager/StagingManager tag、logcat -b events 的 boot_progress 刻度尺（第十章 §10.5）。

场景一：覆盖安装报 INSTALL_FAILED_UPDATE_INCOMPATIBLE

症状：`adb install -r app.apk` 报 "INSTALL_FAILED_UPDATE_INCOMPATIBLE: Package com.example signatures do not match previously installed version; ignoring!"。

定位命令：`dumpsys package com.example | grep -A6 signatures` 取现有签名指纹；`pm path com.example` 看当前 APK 落点（system 分区还是 /data/app）；`apksigner verify --print-certs app.apk` 对比本地新包证书；`logcat -s PackageManager` 看校验段日志。

根因：新旧版本签名指纹不一致，升级一致性校验拒绝（第十一章 §11.4 Prepare 相；对比逻辑见 PackageManagerServiceUtils.compareSignatures:354）——常见于 debug 签名覆盖 release 预装、平台密钥更换未走 v3 密钥轮换。

修复：测试场景先 `pm uninstall` 再装；预装场景必须回到原签名重签；密钥轮换用 v3 scheme 携带 proof-of-rotation 旧签名链（第九章 §9.1），别绕过——绕过即信任链断裂。

场景二：开机慢——扫描段耗时异常

症状：开机 20 秒以上；`logcat -b events` 里 boot_progress_pms_system_scan_start(:1939) 到 boot_progress_pms_ready(:2134) 差值异常大；logcat 出现 "Time to scan packages: 25.1 seconds"。

定位命令：`adb logcat -b events | grep boot_progress` 四事件分段（第十章 §10.5）；`adb logcat -s PackageManager` 看 "Finished scanning system apps. Time: X ms, packageCount: N, cached: M"（InitAppsHelper.java:225）；perfetto 搜 scanDirTracedLI/scanApexPackagesTraced 帧段；`adb shell ls /system/app /system/priv-app | wc -l` 数包。

根因：三类——包数量膨胀（同包名多分区重复预装、废弃 overlay）；首启预编译产物拷贝（requestCopyPreoptedFiles:1927，DexOptHelper.java:725）叠加解析；APEX 模块更新触发 SCAN_DROP_CACHE 丢缓存（第十三章 §13.3 注释③）。

修复：清理重复/废弃预装（一个包名只留一个分区）；确认 /system 预编译 odex/vdex 与镜像一致（减少首启拷贝）；userdebug 才有 PackageCacher 缓存，量产机以减包为主要杠杆；用 bootstat 跟踪 OTA 前后基线。

场景三：应用升级后运行时权限"丢失"或首次使用弹复核

症状：应用更新后相机/定位被拒，或首次使用弹"需要权限复核"；logcat 出现 "Unknown permission android.permission.XXX in package com.example"。

定位命令：`dumpsys package com.example | grep -B2 -A20 "runtime permissions"` 看授予与 flags；`appops get com.example` 看即时开关层；userdebug 下 `cat /data/system/users/0/runtime-permissions.xml | grep -A3 com.example`；`logcat -s PermissionManager PackageManager` 看 restorePermissionState 的 DEBUG_PERMISSIONS 输出。

根因：targetSdk 越过 23 分水岭触发 REVIEW_REQUIRED+REVOKED_COMPAT（PermissionManagerServiceImpl.java:2620-2628）；shared-user 兄弟包升级触发 revokeUnusedSharedUserPermissions 回收无人请求的权限（§12.4 注释⑥）；换 ROM 后权限定义不存在（"Unknown permission"即 restorePermissionState :2681 的跳过日志）；OTA 权限指纹过期触发全量重授但 DefaultPermissionGrantPolicy 例外表未覆盖（第十章 §10.4 注释⑦）。

修复：应用侧实现运行时权限请求并测试 targetSdk 升级矩阵；系统侧把预装应用补进 etc/default-permissions/*.xml；shared-user 拆分前先审计权限并集；用 `pm reset-permissions` 复现重演路径验证修复。

场景四：staged APEX 会话不生效或重启循环

症状：`pm get-stagedsessions` 显示会话长期 STAGED_NOT_READY，或提交后设备重启陷入循环；logcat 出现 "apexd verification failed" 或 apexd revert 记录。

定位命令：`pm get-stagedsessions` / `pm list staged-sessions --only-ready`（PackageManagerShellCommand.java:333/3937）；`logcat -s StagingManager PackageSessionVerifier` 看预重启验证四步（checkActiveSessions/checkRollbacks/checkOverlaps/verifyApex，PackageSessionVerifier.java:187-260）；`dumpsys package` 的 ApexManager 段对照激活版/出厂版（ApexManager.java:1258）。

根因：预重启验证失败——APEX 签名与平台/出厂证书不符、模块间版本依赖不满足（minSdkVersion 门禁，§13.2 注释④）、同模块多会话互斥；apexd 激活失败自动 revert（hash 校验不过、依赖模块版本低于请求）；checkpoint 创建失败。

修复：修正模块签名与依赖版本；先 `pm abandon <sessionId>` 清旧会话再提交（活动会话互斥，§13.4 注释①）；用 `pm get-stagedsessions` 确认 sessionReady 后再重启；激活失败看 apexd 日志的 revert 原因并核对 /data/apex 的 staging 文件完整性。

场景五：安装成功但首启极慢/全程解释执行（dexopt 缺失）

症状：安装无报错，应用首启卡顿数秒；`pm path` 指向的目录下 oat/<abi>/ 缺 odex/vdex 或为空。

定位命令：`pm path com.example` 后 `ls -la $(dirname $(pm path com.example | sed 's/package://'))/oat/*`；`pm art dump`（13 的 artd/策略视图）与 `dumpsys package com.example | grep -i dexopt`；`logcat -s PackageManager dex2oat` 看编译失败原因；`dumpsys jobscheduler | grep -A3 dexopt` 看后台 Job（JOB_IDLE_OPTIMIZE=800）状态。

根因：安装期 dexopt 失败不阻断安装（executePostCommitSteps 源码注释 "don't fail application installs if the dexopt step fails"，InstallPackageHelper.java:2216 起）——空间不足或编译器异常被吞；OTA 后清代码缓存但 profile 缺失导致全量编译慢；后台 dexopt Job 被电量/温度中止（STATUS_ABORT_BATTERY/STATUS_ABORT_THERMAL，BackgroundDexOptService.java:92-101）。

修复：释放空间后 `pm compile -m speed-profile -f com.example` 手动补编；确认设备空闲+充电让 JOB_IDLE_OPTIMIZE 补跑；OTA 场景核对 profile 未被误清（构造器 :2091 的 FLAG_CLEAR_APP_DATA_KEEP_ART_PROFILES 语义，第十章 §10.2 注释⑪）。

场景六：packages.xml 损坏——应用"集体消失"或状态回滚

症状：开机后部分应用消失或授予态重置；logcat 出现 "Need to read from backup settings file" 或 "Cleaning up settings file"。

定位命令：`ls -l /data/system/packages.xml*` 对比主文件与 backup 的时间戳大小；userdebug 下 `head -30 /data/system/packages.xml` 查 XML 完整性；`logcat -s PackageManager` 看 readLPw 的 mReadMessages 摘要。

根因：写账瞬间掉电/存储异常导致主文件不完整——readLPw 回退 backup 并删除可疑主文件（Settings.java:2938-2951，第十二章 §12.2 注释①）；backup 亦过期时，上次写账后的变更（新装/授权）丢失，扫描对账后按盘上 APK 重建。

修复：信任对账自愈（第一章 §1.1 两本账机制），不要手工改 XML；若频繁出现，排查存储健康度与触发 writeSettingsLPrTEMP 的高频调用方（异常循环写账）；量产验证要覆盖"写账中途断电"压力项。

→ 深入：场景一/五的管线细节在第十一章，场景三的账本在第十二章，场景四的双轨在第十三章，场景二/六的开机与对账在第一、十章。

附录 E：PMS 核心机制深度速查表

机制	一句话	源码锚点	所在册
开机扫描	framework→priv-app→app→vendor→data 顺序=信任级	InitAppsHelper:308-342	本册
两本账	内存 mPackages + 磁盘 packages.xml（对账自愈）	Settings.java:2938	本册

机制	一句话	源码锚点	所在册
Computer 快照	不可变查询对象，写后原子换引用	pm/Computer	本册
会话化安装	staging 隔离+原子提交（commit:1638）	PackageInstallerSession	本册
dexopt	13 起 artd 执行（ART APEX），PMS 只管策略	pm/ArtManagerService	本册
签名 v1-v4	JAR→SigningBlock→密钥轮换→fs-verity	ApkSignatureVerifier	本册
Intent 匹配	action→category→data→排序 四关	IntentResolver.java:448	本册
权限三层	声明→授予（运行时）→AppOps（即时开关）	PermissionManagerService:226	本册
RRO 资源覆盖	OverlayManagerService 运行时叠加	om/OverlayManagerService	本册/册1
备份三通道	K/V / Auto / D2D	services/backup/	本册
多用户	包本体一份，PackageUserState per-user	Settings	本册
priv-app 特权	分区+privapp-permissions 白名单 双条件	DefaultPermissionGrantPolicy	本册
回滚	RollbackManager 记录旧版 staging，崩溃自动回退	pm/rollback/	本册
adoptable	SD 卡并入加密池变内部存储	vold VolumeManager	本册/册11
构造器三段式	装配→对账扫描→收官，四个 boot_progress 刻度	PackageManagerService.java:1704-2255	本册第十章
systemReady 十件事	AppsFilter/对账/重授/staged 恢复/后台 dexopt	PackageManagerService.java:3993-4156	本册第十章
mIsUpgrade 差异化	OTA 后重授权限/清代码缓存/promote/埋点	PackageManagerService.java:1954 起	本册第十章
会话三阶段	封印→stream 校验→verify 广播→install 四相	PackageInstallerSession.java:1638→:1828→:2121	本册第十一章
CAS 提交位	mCommitted compareAndSet，2PC 的 PREPARE 标志	PackageInstallerSession.java:1867	本册第十一章
installPackagesLI 四相	Prepare/Scan/Reconcile/Commit+收尾 dexopt	InstallPackageHelper.java:911/:2216	本册第十一章

机制	一句话	源码锚点	所在册
三本账	packages.xml/package-restrictions.xml/runtime-permissions.xml	Settings.java:1430/:1437/:2403	本册第十二章
授予重演	从 manifest+权限定义重推授予态	PermissionManagerServiceImpl.java:2496	本册第十二章
权限写延迟合并	300ms+30% 抖动、上限 2s、指纹升级检查	Settings.java:5373-5497	本册第十二章
APEX 双轨	apexd 管挂载激活，ApexManager 管账本翻译	ApexManager.java:565/:843	本册第十三章
APEX 派生分区	挂载点翻译成 ScanPartition，SCAN_AS_APK_IN_APEX	InitAppsHelper.java:143-178	本册第十三章
staged APEX 闭环	预重启验证→checkpoint→重启激活→systemReady 收尾	PackageSessionVerifier.java:187→PIS:326	本册第十三章